

HiDiTingV100 三方应用

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档介绍了第三方应用的适配和开发流程，用于指导用户快速进行三方应用的开发。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危

符号	说明
	害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 支付宝.....	1
1.1 概述.....	1
1.2 支付码.....	2
1.2.1 方案概述.....	2
1.2.2 Alipay-SDK 和 eSE SDK 接口集成.....	3
1.2.2.1 eSE 集成.....	3
1.2.2.2 Alipay-SDK 集成.....	6
1.2.3 UI 应用开发.....	12
1.2.3.1 绑定流程.....	12
1.2.3.2 支付流程.....	14
1.2.3.3 设置界面.....	15
1.2.3.4 帮助界面.....	16
1.2.3.5 功能列表.....	17
1.3 乘车码.....	17
1.3.1 方案概述.....	17
1.3.2 穿戴设备端适配指导.....	18
1.3.2.1 vendor_file 适配.....	18
1.3.2.2 alipay_net_kal 适配.....	19
1.3.3 穿戴设备端 UI 应用开发.....	20
1.3.4 手机端 APP 开发指导.....	24
1.3.4.1 概述.....	24
1.3.4.2 功能描述.....	24
1.3.4.3 手机侧 socket 示例代码讲解.....	24

1.3.4.3.1 socket 类成员.....	24
1.3.4.3.2 构造函数.....	26
1.3.4.3.3 类属性设置函数.....	27
1.3.4.3.4 socket 连接调用接口.....	27
1.3.4.3.5 socket 发送数据调用接口.....	27
1.3.4.3.6 socket 数据接收接口.....	28
1.3.4.3.7 关闭客户端 socket 接口	28
1.3.4.3.8 DNS 域名解析功能.....	28
1.3.4.4 蓝牙指令&socket 交互示例代码讲解.....	29
1.3.4.4.1 网络状态更新交互指令（手机->单板）	32
1.3.4.4.2 DNS 域名解析帧交互指令（单板->手机）	35
1.3.4.4.3 建立 TCP 连接指令（单板->手机）	38
1.3.4.4.4 socket 数据读取.....	42
1.3.4.4.5 TCP 响应数据指令（手机->单板）	43
1.3.4.4.6 TCP 上网请求的数据指令（单板->手机）	60
1.3.4.4.7 关闭 socket 蓝牙指令处理流程（单板->手机）	62
1.3.4.4.8 收到指令成功应答帧处理逻辑.....	63
1.4 手机端数传适配指导.....	65
1.4.1 蓝牙数传入口 Activity 页面.....	65
1.4.1.1 蓝牙 SPP 数传 Activity 调用示例.....	65
1.4.1.2 蓝牙 BLE 数传 Activity 调用示例	65
1.4.1.3 数传上下行页面适配器示例.....	66
1.4.2 蓝牙上传场景代码调用说明.....	67
1.4.2.1 定义蓝牙文件上传回调函数.....	67
1.4.2.2 实例化蓝牙文件上传类.....	70
1.4.2.3 实现蓝牙接收数据接口	71
1.4.2.4 停止蓝牙文件上传接口	72
1.4.3 蓝牙下载场景代码调用说明.....	72
1.4.3.1 定义蓝牙文件下载回调函数.....	72
1.4.3.2 实例化蓝牙文件下载类.....	72
1.4.3.3 实现蓝牙接收数据接口	74

1.4.3.4 停止蓝牙文件下载接口	75
1.4.4 蓝牙查询文件列表接口说明	76
1.4.4.1 定义查询文件列表蓝牙回调函数	76
1.4.4.2 实例化查询文件列表类	78
1.4.4.3 实现蓝牙接收数据接口	78
1.4.5 文件传输帧详解	79
1.4.5.1 传输协商帧	79
1.4.5.1.1 协商帧字段详解	79
1.4.5.1.2 协商帧结构及示例	82
1.4.5.1.3 协商应答帧结构及示例	82
1.4.5.2 传输请求帧	83
1.4.5.2.1 请求帧字段详解	83
1.4.5.2.2 传输请求帧结构及示例	84
1.4.5.3 传输回应帧	84
1.4.5.3.1 传输回应帧字段详解	84
1.4.5.3.2 传输回应帧结构及示例	85
1.4.5.4 传输通知帧	91
1.4.5.4.1 传输通知帧字段详解	91
1.4.5.4.2 传输通知帧结构及示例	92
1.4.5.5 传输停止帧	93
1.4.5.5.1 传输停止帧字段详解	93
1.4.5.5.2 传输停止帧结构及示例	93
1.4.5.6 传输列举帧	94
1.4.5.6.1 传输列举帧字段详解	94
1.4.5.6.2 传输列举帧结构及示例	95
1.4.5.6.3 传输列举应答帧示例	96

插图目录

图 1-1 总体方案框图 2

图 1-2 支付方案框图 3

图 1-3 app 目录结构 8

图 1-4 绑定流程示意 14

图 1-5 解绑流程示意 16

图 1-6 乘车码整体方案框图 17

图 1-7 乘车码流程图 22

图 1-8 异步更新流程图..... 23

图 1-9 蓝牙 diag 指令和手机侧 socket 交互指令流程 29

表格目录

表 1-1 cis*接口说明 4

表 1-2 驱动接口说明 4

表 1-3 封装 alipay-SDK 的接口说明..... 9

表 1-4 vendor_file api 18

表 1-5 alipay_net_kal api..... 19

表 1-6 socket 类成员变量含义 24

表 1-7 手机给手表发送网络状态指令字段描述 32

表 1-8 蓝牙 SPP 发送网络请求报文示例 32

表 1-9 域名解析字段描述 35

表 1-10 蓝牙 SPP 域名解析请求帧结构示例..... 35

表 1-11 域名解析应答帧示例..... 36

表 1-12 建立连接字段描述 38

表 1-13 蓝牙 SPP 建立连接请求帧结构示例..... 40

表 1-14 建立连接响应帧结构示例 40

表 1-15 服务器 TCP 响应数据字段描述 43

表 1-16 蓝牙 SPP 回复服务器 TCP 响应不需要分帧报文示例..... 45

表 1-17 TCP 响应数据帧字段描述 49

表 1-18 蓝牙 SPP 起始报文示例 52

表 1-19 蓝牙 SPP 过程报文示例 1 54

表 1-20 蓝牙 SPP 过程报文示例 2 55

表 1-21 蓝牙 SPP 结束报文示例	56
表 1-22 TCP 上网请求字段描述	60
表 1-23 蓝牙 SPP 发送 TCP 请求报文示例	61
表 1-24 服务器 TCP 响应数据字段描述	62
表 1-25 蓝牙 SPP 发送关闭 socket 请求帧示例	63
表 1-26 SPP 数传参数说明	65
表 1-27 BLE 数传参数说明	66
表 1-28 上传下载页面说明	66
表 1-29 上传下载页面传参说明	67
表 1-30 蓝牙文件上传回调函数接口说明	68
表 1-31 蓝牙文件上传类参数说明	70
表 1-32 蓝牙文件下载类参数说明	73
表 1-33 查询文件列表蓝牙回调函数接口说明	77
表 1-34 查询文件列表类参数说明	78
表 1-35 传输协商帧字段描述	79
表 1-36 传输下载协商帧结构示例	82
表 1-37 传输下载协商应答帧结构示例	83
表 1-38 传输请求帧字段描述	83
表 1-39 传输下载协商帧结构示例	84
表 1-40 传输回应帧字段描述	84
表 1-41 传输回应帧示例	86
表 1-42 传输通知字段描述	91
表 1-43 传输通知帧示例	93
表 1-44 传输停止帧字段描述	93
表 1-45 传输停止帧示例	94
表 1-46 传输列举帧字段描述	94
表 1-47 传输列举帧示例	95

表 1-48 传输列举应答帧示例 96

Draft

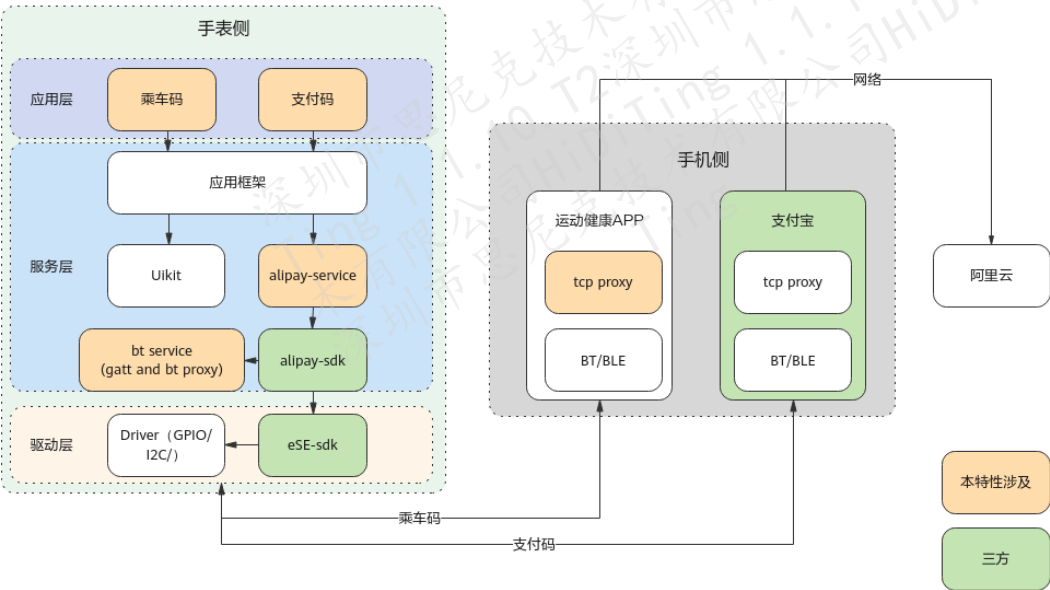
1 支付宝

- 1.1 概述
- 1.2 支付码
- 1.3 乘车码
- 1.4 手机端数传适配指导

1.1 概述

本章主要介绍基于支付宝 eSE 安全加密芯片方案、适配支付宝侧 SDK、支付码和乘车码的应用开发流程以及硬件要求限制，指导用户进行支付宝应用开发，总体方案如图 1-1 所示。

图1-1 总体方案框图



从整体处理流程上看，支付码、乘车码特性主要涉及以下几个部分：UI 应用界面、Alipay SDK 集成、eSE SDK 集成、Alipay 服务、BT Proxy 服务。

说明

支付码、乘车码属于三方应用对接，本章只基于 UIKit 实现 Sample，正式商用前客户需要准备样机联系支付宝，提前配置相关账户信息，调试完成后需要提供支付宝进行送测。

1.2 支付码

1.2.1 方案概述

本章简单介绍阿里支付宝团队的加密芯片方案，通过加密芯片，实现支付资产安全隔离和安全存储保护要求，同时利用安全芯片的唯一 HUK 进行授权码验证，确保支付资产和安全芯片——绑定，保证支付安全，支付整体方案框图如图 1-2 所示。

图1-2 支付方案框图



须知

支付特性穿戴设备端硬件准入限制：

- 显示屏：支持屏幕背光，圆形屏幕分辨率不低于 320×320，矩形屏幕分辨率不低于 280×280。
- 设备需保证时钟的准确性，确保一年内时间偏移不超过 60s，并且拥有对时功能。
- ROM：代码段 200KB。
- RAM：7KB 栈空间，8KB 堆空间，共 15KB，部署在 psram 上。

1.2.2 Alipay-SDK 和 eSE SDK 接口集成

通过从平头哥芯片社区下载相关代码 [可穿戴支付安全解决方案-平头哥芯片开放社区-平头哥 \(t-head.cn\)](#)，当前发布 SDK 下载使用是 Alipay SDK v3.0.1，用户使用根目录“vendor/alipay_sdk”即可，无需额外下载。

资源名称	更新时间	大小	操作
Alipay SDK v3.0.1 (乘车码/支付码低功耗版本)	2023-10-26 11:46:48	13.84MB	点击下载
Alipay SDK v2.0.2 (支付码低功耗版本)	2023-10-17 14:02:03	15.53MB	点击下载
Alipay SDK v1.1.4 (支付码非低功耗版本)	2023-10-17 14:01:45	14.57MB	点击下载

1.2.2.1 eSE 集成

适配工作主要分为三个步骤：

- 步骤 1 集成 “vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\soc” 目录下的样例代码，并适配修改 csi*接口的实现。

可参考 sdk 目录下 “application\wearable\alipay\driver_adapt\vendor_product.c” 的实现，如表 1-1 所示对接口简单说明。

表1-1 cis*接口说明

函数名称	功能说明
csi_get_deviceid	获取设备蓝牙地址和长度，格式为 “AA:BB:CC:00:11:22”。
csi_get_timestamp	获取当前系统时间戳（Unix 时间戳格式）。
csi_get_sn	获取设备 SN，参考 csi_get_deviceid 获取蓝牙地址实现即可。
csi_get_companyname	获取设备公司名称，公司名称需要提供给支付宝进行配置。
csi_get_productmodel	获取设备产品名称，产品名称需要提供给支付宝进行配置。
csi_ble_write	发送蓝牙数据接口。
csi_log	日志打印接口，可通过日志等级控制。
csi_log_ext	日志打印接口。
csi_malloc	内存申请接口。
csi_calloc	内存申请接口，并初始化。
csi_free	内存释放接口。
csi_realloc	调整内存空间接口。

步骤 2 驱动接口适配。

可参考 sdk 目录下 “application\wearable\alipay\driver_adapt\soc_adapt.c” 的实现，如表 1-2 所示对接口简单说明。

表1-2 驱动接口说明

函数名称	功能说明
------	------

函数名称	功能说明
IIC_Master_Init	IIC master 初始化接口。
IIC_Master_Send	IIC master 发送接口。
IIC_Master_Receive	IIC master 接收接口。
Delay_Ms	延迟一段时间接口，单位：ms。
Set_GPIO_State	设置 GPIO 管脚输出高低状态接口。

说明

其中用户需要在 “drivers\boards\brandy_evb\product\product.h” 文件中配置 IIC 选择和低功耗管脚，可参考：

```
#define ALIPAY_SEC_I2C_INDEX          I2C_BUS_3
#define ALIPAY_SEC_GPIO              S_AGPIO_L29
```

步骤 3 完成 test 目录下的测试案例。

须知

在产品制造过程中，默认情况下安全凭证固化在加密芯片里边。但是由于某些原因，出厂的加密芯片烧写凭证不完整或损坏时，如果发现不及时，整个产品生产都要返工重新制造。为防止该类问题，需要在产线检测 eSE 芯片安全凭证的正确性和完整性。

- 使用 hal_get_credential 可用于验证安全凭证是否完整。
- 使用 hal_compat_check 可用于验证安全凭证是否和硬件绑定。

可参考 “vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\soc\test\test_api.c”，目前 SDK 也提供了 “AT^ALIPAYVERIFY” 可参考作为产线测试项。

eSE 芯片提供了 yunit 测试框架，注册了 23 个测试用例，目前 sdk 提供了 AT 命令接管测试入口：AT^ALIPAYSECTEST

执行完成后，显示如下，则为成功：

```
APP|[SEC PAY E/] yunit_test_print_result:
-----
APP|[SEC PAY E/] yunit_test_print_result: 23 test cases, 23 success, 0 failed
```

23 个用例可在 “vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\src\test\test_api.c” 文件中查看 “api_record_test_entry” 函数：

```
yunit_add_test_case(suite, "test_get_compile_timestamp",  
case_test_get_compile_timestamp);  
  
yunit_add_test_case(suite, "test_get_productmodel", case_test_get_productmodel);  
  
yunit_add_test_case(suite, "test_memory_align", case_test_memory_align);  
  
yunit_add_test_case(suite, "test_libc_function", case_test_libc_function);  
  
yunit_add_test_case(suite, "test_int_endian", case_test_int_endian);  
  
yunit_add_test_case(suite, "hal_get_credential", case_test_get_credential);  
  
yunit_add_test_case(suite, "hal_compat_check", case_test_compat_check);  
  
yunit_add_test_case(suite, "hal_suid_update", case_test_hal_suid_update);  
  
yunit_add_test_case(suite, "hal_suid_remove", case_test_hal_suid_remove);  
  
yunit_add_test_case(suite, "hal_mem", case_test_hal_mem);  
  
yunit_add_test_case(suite, "hal_sha", case_test_hal_sha);  
  
yunit_add_test_case(suite, "hal_aes", case_test_hal_aes);  
  
yunit_add_test_case(suite, "hal_asset", case_test_hal_asset);  
  
yunit_add_test_case(suite, "hal_random_pressure",  
case_test_hal_random_pressure);  
  
yunit_add_test_case(suite, "hal_mcu_pressure", case_test_hal_mcu_pressure);  
  
yunit_add_test_case(suite, "hal_random", case_test_hal_random);  
  
yunit_add_test_case(suite, "hal_uuid", case_test_hal_uuid);  
  
yunit_add_test_case(suite, "hal_sn", case_test_hal_sn);  
  
yunit_add_test_case(suite, "hal_deviceid", case_test_hal_deviceid);  
  
yunit_add_test_case(suite, "get_fw_version", case_test_get_fw_version);  
  
yunit_add_test_case(suite, "hal_companyname", case_test_hal_companyname);  
  
yunit_add_test_case(suite, "hal_timestamp", case_test_hal_timestamp);  
  
yunit_add_test_case(suite, "hal_prototype", case_test_hal_prototype);
```

----结束

1.2.2.2 Alipay-SDK 集成

适配工作主要分为三部分：

1. 提供编译工具链给支付宝侧，集成返回的支付 SDK 库。


```
— 3rdparty
— app
— CMakeLists.txt
— fwk
— hal
— share
```

支付对外提供的接口都是在 app 里面；3rdparty 为三方库；fwk 为提供的组件模块，可查看 depends.txt 按需集成。

主要关注 app 目录，其中：

- aid_code：帮助码模块。
- bind_ble：蓝牙绑定模块。
- pay_code：支付模块。
- setting：账户设置模块。
- trans_code：乘车码模块。
- pay_result：不支持暂不用关注。
- provision：暂不使用暂不用关注。

app 目录结构如图 1-3 所示。

图1-3 app 目录结构



每一个 app 目录下面都会有对外头文件、库文件以及 depends.txt，depends.txt 描述了依赖的组件(fwk, hal, 3rdparty)。

2. alipay service 开发

alipay service 主要实现了对 Alipay-SDK 的接口封装、service 初始化、消息线程创建等功能，可参考 “application\wearable\alipay\service\alipay_feature.h” 头文件查看函数说明。

通过 alipay_svr_get_api 可以获取支付宝提供的 app 功能接口，也可直接调用 Alipay-SDK 的接口，其封装后函数结构体如表 1-3 所示。

表1-3 封装 alipay-SDK 的接口说明

结构体成员函数	功能说明
set_bt_broadcast	发起广播或停止广播。
get_logon_id	获取登录账号 ID。
get_nickname	获取登录账号昵称。
get_bind_state	获取是否已经绑定，进入应用通过此接口判断是否已经绑定过。
get_bind_status	获取当前绑定状态，扫码绑定时用于判断当次绑定过程的状态。
bind_start	初始化 Alipay SDK，并通知 BT 开始发起广播。
bind_finish	绑定完成。
get_bind_str	获取绑定码，用于生成绑定二维码。
get_pay_str	获取付款码，用于生成付款二维码和条形码。
get_aid_str	获取帮助码，用于生成帮助二维码。
remove_bind	解绑接口。
proc_packet	接收手机端蓝牙消息函数，传递至 Alipay-SDK。
exit_lpm	eSE 退出低功耗接口。
enter_lpm	eSE 进入低功耗接口。
get_last_transcode	获取最近一次使用的乘车码信息，断电后会丢失，注意不是字符串。
get_local_translist	获取本地缓存的开通乘车的城市列表。

结构体成员函数	功能说明
get_online_translist	获取服务端的城市列表。
check_trans_cartstatus	用于检查指定乘车卡的有效性。
update_trans_cardinfo	更新指定乘车卡的数据。
get_transcode	获取指定卡的乘车码码流，注意不是字符串。

以上为封装 Alipay-SDK 的接口部分，除此 serive 还需要提供初始化函数接口 uapi_alipay_svr_init。该接口需完成对 eSE 的 HS_IIC_Init 接口初始化、低功耗 RST 管脚初始化、以及线程创建等功能。

说明

由于支付宝特性依赖于 eSE 芯片对接，所以当前 uapi_alipay_svr_init 未加入到系统初始化流程中，故初始化需要执行 AT 命令：AT+ALIPAYDEBUG，用户可根据需求进行初始化。

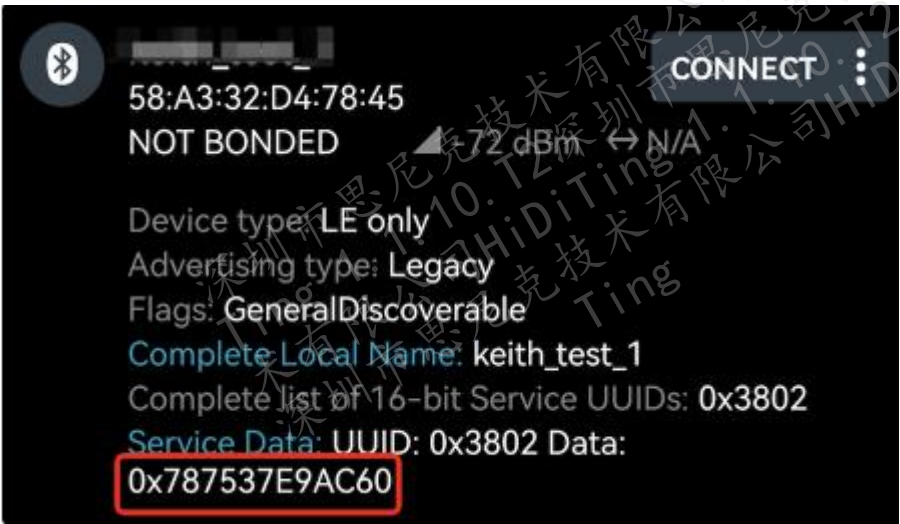
3. alipay bluetoooh 模块开发

支付宝蓝牙配置要求如下，可参考

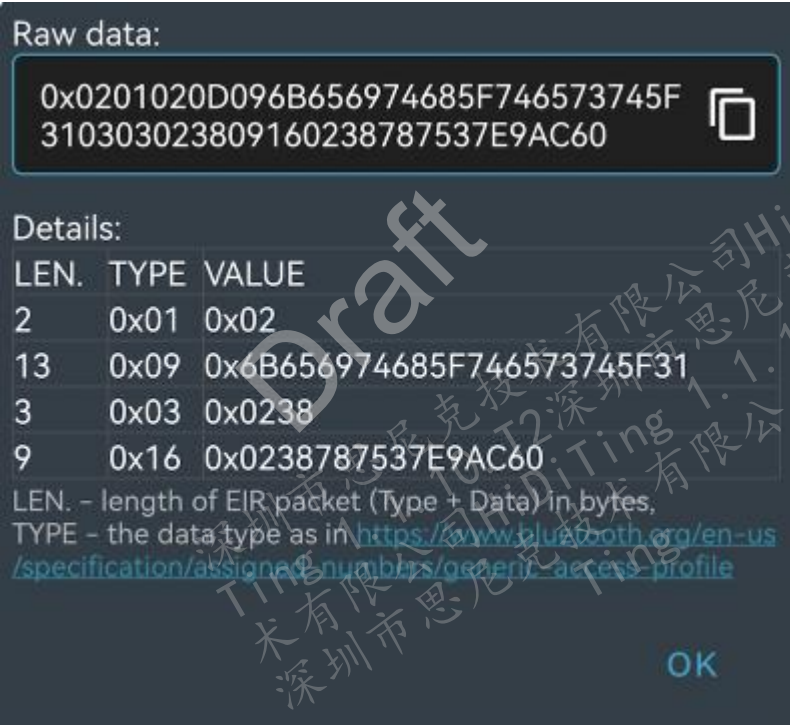
“vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\doc” 目录下的“支付宝智能穿戴带乘车码 SDK 对接流程.pdf”。

- 约定设备端支付宝的蓝牙Service UUID： "00003802-0000-1000-8000-00805f9b34fb"
- 对应的 CHARACTERISTIC UUID： "00004a02-0000-1000-8000-00805f9b34fb"
- CHARACTERISTIC 权限： PROPERTY_WRITE + PROPERTY_READ + PROPERTY_NOTIFY

由于 IOS 获取设备端 BLE MAC 地址的需求，需要在广播包的上述 Service UUID 的 service data 中填入MAC 地址的 byte 数组，让手机端可以找到对应设备，两次广播最大间隔时间不超过 2 秒。如下图所示：



广播数据如下图所示：



并在该 CHARACTERISTIC 下添加 descriptors，对应的 UUID：00004b02-0000-1000-8000-00805f9b34fb。

读取对应的 Value：MAC 地址的 byte 数组。如下图所示：



具体实现可参考 “application\wearable\alipay\service\alipay_bluetooth.c” 实现。

1.2.3 UI 应用开发

支付应用主要涉及四个部分，即绑定、支付、设置、帮助界面，当前 SDK 仅支持离线支付，即穿戴设备端不会获取支付状态，由商家或者手机端感知支付结果。

UI 界面需要严格按照支付宝设计稿实现，后续调试完成需要向阿里侧演示验收，设计流程和 UI 资源可参考 “vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\参考视觉稿\成人-手表手环-视觉稿-带流程.sketch”。

SDK 提供基于 UIKit 的 native 应用 sample，LVGL 需要用户自行开发，可参考 “application\wearable\nativeapp\nativeui\alipay”。

1.2.3.1 绑定流程

绑定流程主要界面为如下 5 个：

- 绑定界面：



- 绑定码-绑定码获取成功页面；
- 二维码编码规范，纠错等级H，其他要求详见《接入码规范》；
- 码串demo：ALIPAY_WATCH#2#12:34:56:78:8A:BC#CR.SB.LY.alipay-demo#4.4.0.169457

- 绑定状态界面（动态）：



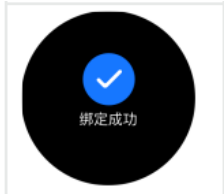
- 绑定码-正在绑定页面；
- 动画效果，顺时针转动，2秒1圈；
- 加载动画可以用自带的类似的动画替代；

正在绑定滑动跳转界面



- 绑定码-退出提醒页面；
- 在该页面停留时要求绑定仍正常进行，如果此时绑定成功了则直接跳转「绑定码-绑定成功」，绑定失败了则直接跳转「绑定码-绑定失败」；
- 点击「退出」，退出支付宝；
- 点击「去绑定」，回到「绑定码-正在绑定」；

绑定成功视图



- 绑定码-绑定成功页面；
- 停留3秒，自动跳转至「功能选择页」；

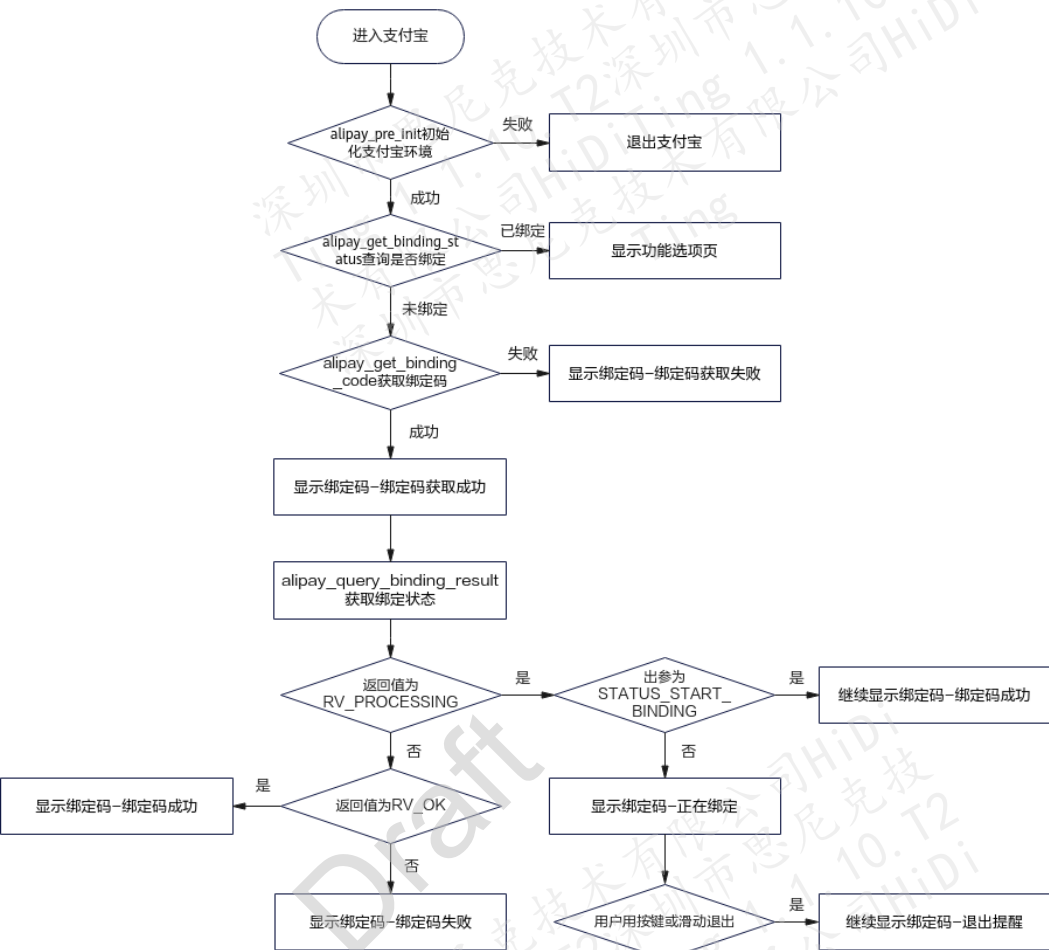
绑定失败视图



- 绑定码-绑定失败页面；
- 停留8秒，自动跳转至绑定码页面；

绑定流程如图 1-4 所示。

图1-4 绑定流程示意



1.2.3.2 支付流程

支付流程主要页面说明：

• 支付-二维码



- 付款码_二维码页；
- 二维码编码规范，纠错等级Q，其他要求详见《支付宝智能穿戴-码规范》；
- 码串demo：208812345678901234；
- 进入付款码后优先展示二维码；
- 点击码重新刷新码值；
- 在该页面停留时，每60秒刷新一次码值；
- 熄屏重新亮屏后，如果还在该页面需要刷新一次码值；

• 支付-条形码



- 付款码_条形码页；
- 编码格式为Code 128，建议使用Code128Auto，其他要求详见《支付宝智能穿戴-码规范》；
- 点击码重新刷新码值；
- 在该页面停留时，每60秒刷新一次码值；
- 熄屏重新亮屏后，如果还在该页面需要刷新一次码值；

支付流程较简单，通过 `alipay_get_paycode` 获取支付码串，绘制二维码、条形码界面，支持滑动切换和按键切换。

支持点击二维码界面和条形码界面和定时刷新，sample 没有做亮屏刷新界面，如需用户可自行实现此功能。

1.2.3.3 设置界面

设置界面如下：主要是用于用户解绑。

- 设置-信息页面



- 设置-信息页面；
- 昵称和账号长度超出屏幕范围，可以用滚动显示；
- 账号长度超出屏幕范围，可以用滚动显示，或者隐藏中间的字符，比如 396...@163.com；
- 昵称中出现不支持的字符要求用圆角的正方形（U+25A2）代替；
- 点击「解除绑定」跳转到「设置-解绑确认」；

- 设置-解绑确认页面



- 设置-解绑确认页面；
- 点击「确认」，跳转到「设置-解绑后提醒」；
- 点击「取消」，跳转到「设置-信息」；

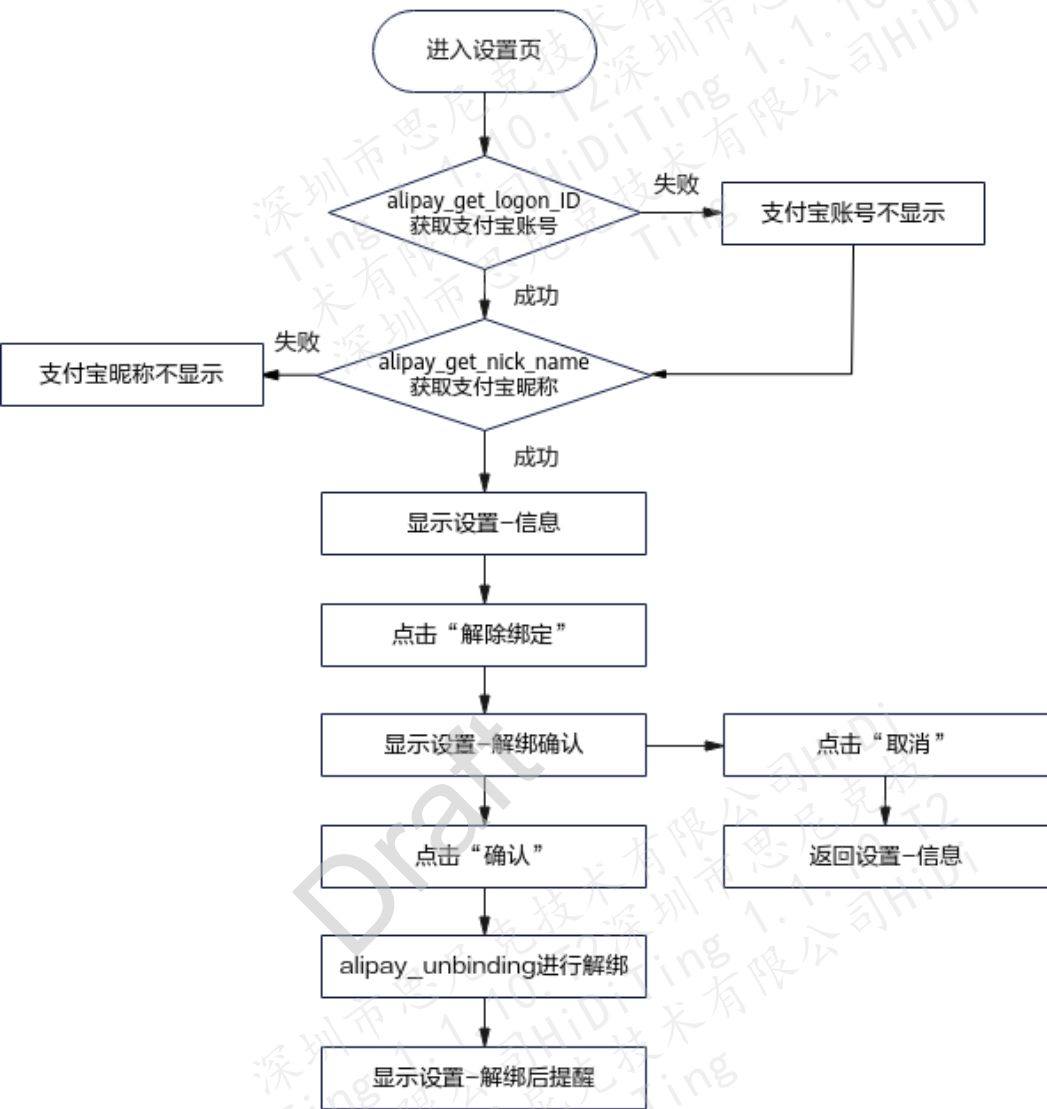
- 设置-解绑后提醒页面



- 设置-解绑后提醒页面；
- 其中的文案「设备侧」需要替换成具体的设备类型，比如「手表侧」、「手环侧」、「学生卡侧」等；
- 点击「确认」，跳转绑定码页面；

解绑流程如图 1-5 所示。

图1-5 解绑流程示意



1.2.3.4 帮助界面

提供帮助码供手机支付宝扫码跳转至手机端帮助界面，视图要求如下：



- 帮助码-获取帮助码成功页面；
- 二维码编码规范，纠错等级L，其他要求详见支付宝智能穿戴-码规范》；
- 码串demo：ALIPAY_HPWT#0#
{ "mac": "12:34:56:78:8A:BC", "gtime": "1658284781", "pmodel": "alipay.demo", "ver": "4.4.0" }

帮助流程较简单，通过 alipay_get_aid_code 获取帮助码串生成二维码，供手机端支付宝扫码。

1.2.3.5 功能列表

手机端支付宝扫绑定码绑定成功后，绑定成功界面三秒后会自动跳转至功能列表界面，如下所示：



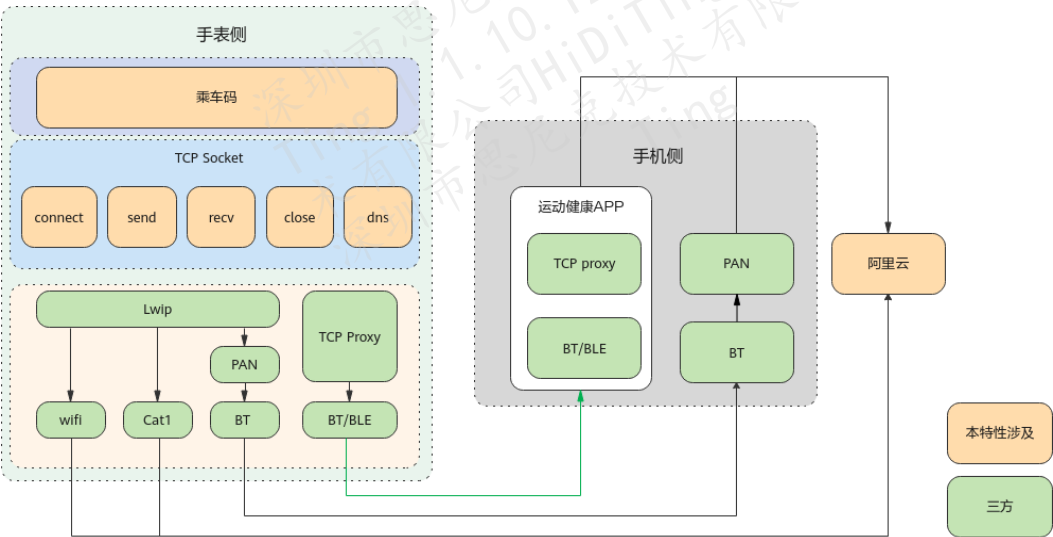
功能列表为 LIST 界面，点击后跳转至对应应用界面。

1.3 乘车码

1.3.1 方案概述

乘车码整体方案如图 1-6 所示。

图1-6 乘车码整体方案框图



HiDiTingV100 使用 TCP Proxy&BT/BLE 方案采用蓝牙做上网代理，使得手表可以通过蓝牙与手机连接时可以借助手机的联网能力，访问网络服务。

该方案需要手机侧 APP 实现 TCP 代理功能，通过手机网络完成与支付宝服务器通讯，获取乘车码相关资源。

HiDiTingV100 SDK 版本没有集成 lwip 能力，最终用户可以根据实际情况适配。

须知

乘车码特性穿戴设备端硬件准入条件：

- 显示屏：屏幕分辨率（DPI）不低于 320×320（圆形）或 280×280（方形）。
- ROM：需要放置至少 800KB 放置乘车码代码（包含 quickjs），部署在 NorFlash 上；1MB 的空间放置数据，部署在 NandFlash 上。
- RAM：需要准备 25KB 栈空间，20KB 的堆空间，共 45KB 的 RAM。

1.3.2 穿戴设备端适配指导

1.3.2.1 vendor_file 适配

vendor_file 主要用于保存乘车码离线资源以及 js 脚本。

表1-4 vendor_file api

函数名	功能说明
alipay_open_rsvd_part	打开文件名为 filename 的文件，要求权限为可读写。
alipay_write_rsvd_part	往文件中写数据。
alipay_read_rsvd_part	从提供给 alipay kv 存储的文件中读取数据。
alipay_close_rsvd_part	关闭打开的文件。
alipay_access_rsvd_part	查看文件是否存在。
alipay_remove_rsvd_part	删除文件。
alipay_clear_rsvd_part	清除所有通过 alipay_write_rsvd_part 存储的 alipay 文件。

适配代码路径：application\wearable\alipay\driver_adapt\vendor_file.c

sdk 自带测试代码路径：

vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\lib\rv70_llvm\hal\file\sample\test_file.c

1.3.2.2 alipay_net_kal 适配

alipay_net_kal 主要适配 TCP 相关接口 (UDP 不支持)。板端使用 diag 协议完成与手机 APK 交互, 实现 TCP 对应接口, 具体帧格式请参见《穿戴类产品对外交互协议》的“阿里乘车码”章节。

表1-5 alipay_net_kal api

函数名	功能说明
alipay_iot_gettimeofday	获取 UTC 时间。
alipay_iot_get_local_time	获取系统当前时间。
alipay_task_sleep	使任务休眠时间, 单位: ms。
alipay_iot_dns	域名解析, 返回 IPv4 地址。
alipay_iot_socket_create	创建套接口。
alipay_iot_socket_connect	向指定地址发起连接。
alipay_iot_socket_write	发送数据。
alipay_iot_fd_isset	检查 fdset 中对应的 fd 是否有被 selected。
alipay_iot_fd_setbit	fdset 置位。
alipay_iot_fd_zero	fdset 清零。
alipay_iot_select	检查 socket 事件。
alipay_iot_socket_read	接收数据。
alipay_iot_socket_close	关闭套接口。

适配代码路径: application\wearable\alipay\driver_adapt\vendor_net.c

sdk 自带测试代码路径:

vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\lib\rv70_llvm\hal\network\sample\test_network.c

1.3.3 穿戴设备端 UI 应用开发

乘车码视图是从功能列表界面跳转，主要包含卡列表页、乘车码页面、未开通引导页、提示页。

UI 界面需要严格按照支付宝设计稿实现，后续调试完成需要向阿里侧演示验收，设计流程和 UI 资源可参考“vendor\alipay_sdk\alipay_sec_sdk_v3.0.0\参考视觉稿\成人-手表手环-视觉稿-带流程.sketch”。

SDK 提供基于 UIKit 的 native 应用 sample，LVGL 需要用户自行开发，可参考“application\wearable\nativeapp\nativeui\alipay”。

主要界面如下：

- 卡列表页



乘车码列表不为空时显示，点击乘车卡跳转对应城市乘车码界面，alipay-SDK 最大支持 20 个数据。

- 乘车码页面



二维码纠错等级 L，不同城市的二维码须保持大小一致；点击“切换交通卡”返回乘车码列表页面。

- 乘车码开通引导页

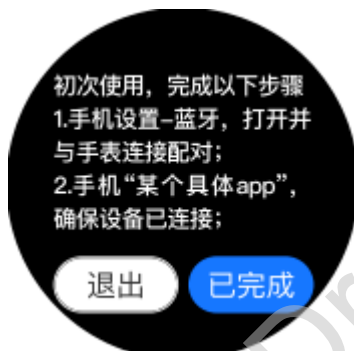


二维码纠错等级 L，手机端支付宝未开通乘车码，乘车码列表为空跳转界面。滑动可返回功能选择页。

未开通引导页码值为固定 url scheme:

alipays://platformapi/startapp?appId=20002047&scene=bus.

- 首次使用引导页



首次使用，点击“退出”返回功能选择页；点击“已完成”从服务端获取乘车码列表。

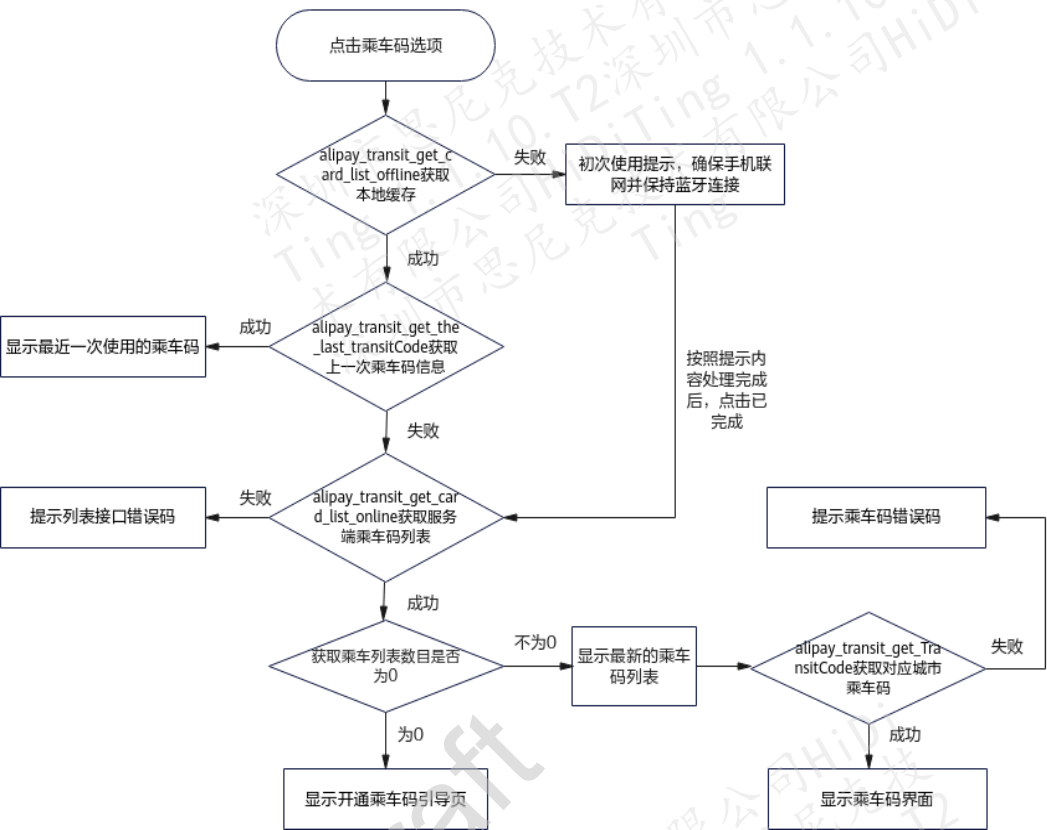
- 错误提示页



错误提示页，根据不同调用接口的返回错误码，跳转至不同的错误提示页，具体参考“成人-手表手环-视觉稿-带流程.sketch”。

乘车码流程如图 1-7 所示。

图1-7 乘车码流程图



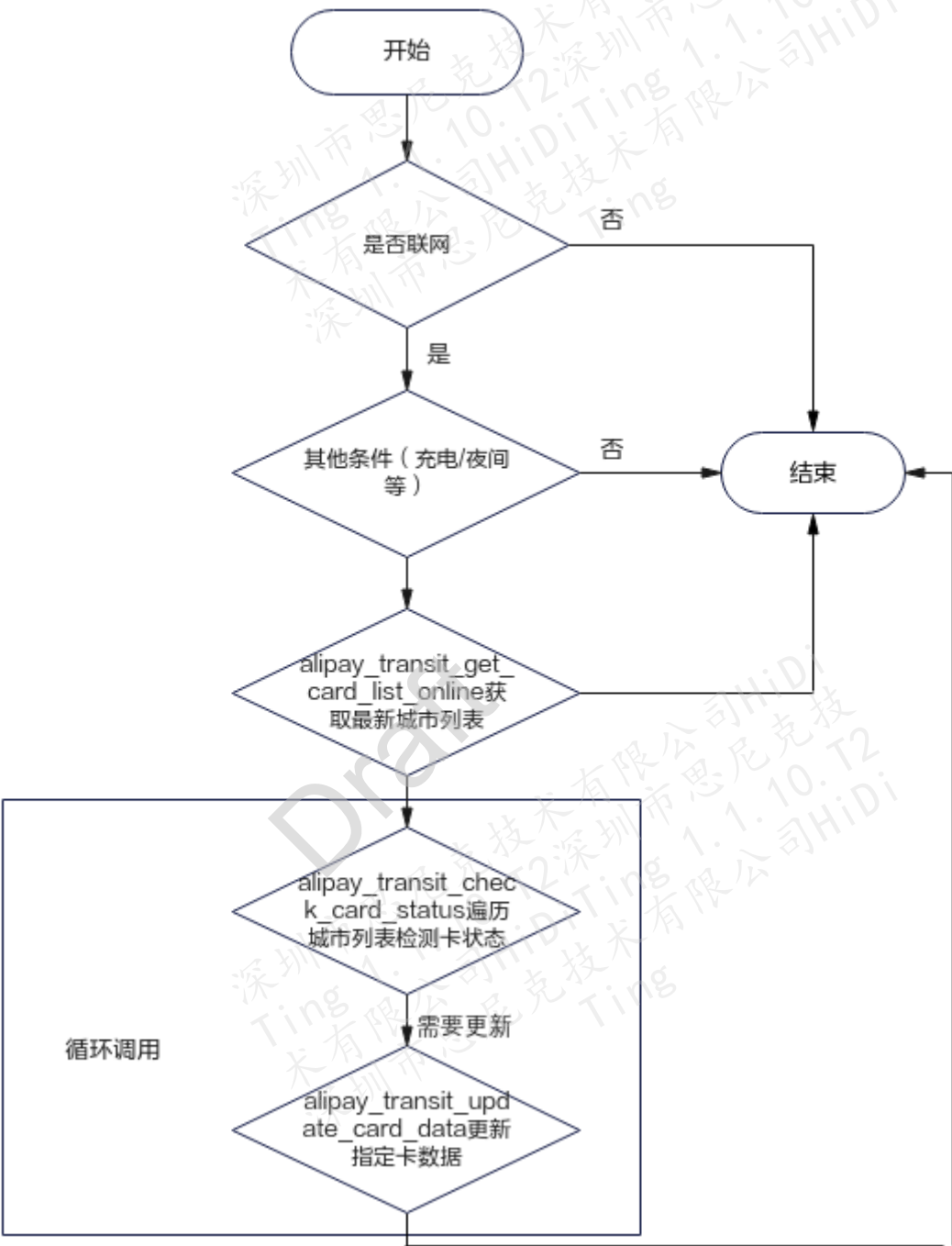
须知

异步更新可以减少蓝牙上网通道状态的不确定性，尽量在适当时机去触发卡列表和卡信息的提前更新，防止在用户实际使用时网络没连或连接不稳定导致的体验和可用性问题。

当前 sample 没有提供异步流程。建议用户根据自身需求，定制异步更新策略。

异步更新流程如图 1-8 所示。

图1-8 异步更新流程图



说明

更新条件:

1. 本地无缓存;
 2. 过期时间在未来 12小时内;
 3. 剩余使用次数低于 4 次;
- 以上条件满足任何一条, 建议触发更新。

1.3.4 手机端 APP 开发指导

1.3.4.1 概述

本章节主要介绍手机侧 APK 适配 socket 代理对接阿里服务器的示例代码指导, **仅用于给客户参考, 不可直接用于商用。**

1.3.4.2 功能描述

- 实现手机侧 socket 的连接、发送指令、接收指令、关闭、DNS 解析功能。
- 实现手机给手表发送网络状态指令请求帧、应答帧。
- 实现接收解析手表给手机发送域名解析请求指令请求帧、发送应答帧。
- 实现接收解析手表给手机发送 connect 指令请求帧、发送应答帧。
- 实现接收解析手表给手机发送 tcp 上网请求的请求帧、发送应答帧。
- 实现手机给手表回复服务器 tcp 响应数据请求帧、应答帧。
- 实现接收解析手表给手机发送 tcp close 指令的请求帧、发送应答帧。

1.3.4.3 手机侧 socket 示例代码讲解

示例代码路径:

WearTest\app\src\main\java\com\wrtm\weartest\serverclient\ClientSocket.java

1.3.4.3.1 socket 类成员

socket 类成员变量含义如表 1-6 所示。

表1-6 socket 类成员变量含义

socket 类成员变量	说明
private Socket clientSocket = null;	socket 对象。
private String serverAddress;	服务器 IP 地址。
private int serverPort;	服务器端口号。

socket 类成员变量	说明
private boolean isLoopReveiveData = true;	是否循环读取数据。
private AliSocketStatusListener onSocketStatusListener;	socket 处理回调函数。

接口定义在 socket 处理回调函数的
“WearTest\app\src\main\java\com\wrtm\weartest\componentinterface\AliSocketStatusListener.java” 里，接口功能如下：

```
public interface AliSocketStatusListener {  
    /**  
     * socket连接成功  
     */  
    void socketClientConnectSuccess();  
  
    /**  
     * socket连接失败  
     */  
    void socketClientConnectFailed();  
  
    /**  
     * socket连接异常  
     */  
    void socketClientConnectException();  
  
    /**  
     * socket通道发送数据成功  
     *  
     * @param msg byte[]  
     */  
    void socketClientSend(byte[] msg);  
  
    /**  
     * socket通道发送数据失败  
     *  
     * @param msg byte[]  
     */  
    void socketClientSendFailed(byte[] msg);  
}
```

```

/**
 * socket通道接收数据成功
 *
 * @param buf 响应内容
 */
void socketClientRecieveSuccess(byte[] buf);

/**
 * socket通道接收数据失败
 *
 * @param msg 失败原因
 */
void socketClientRecieveFailed(String msg);

/**
 * socket通道接收数据异常
 */
void socketClientRecieveException();

/**
 * socket关闭成功
 */
void socketCloseSucess();

/**
 * socket关闭失败
 */
void socketCloseFailed();
}

```

1.3.4.3.2 构造函数

外部类实例化时会生成 socket 对象：

```

public ClientSocket() {
    clientSocket = new Socket();
}

```

1.3.4.3.3 类属性设置函数

- 外部调用：用于设置 socket receive 是否循环接收数据，参数为 true 则 socket 会一直接收数据、参数为 false 则 socket 不会再接收数据。

```
public void setLoopReveiveData(boolean isLoopReveiveData) {  
    this.isLoopReveiveData = isLoopReveiveData;  
}
```

- 外部调用：用于设置 socket 各个状态的回调函数。

```
public void setOnSocketStatusListener(AlisocketStatusListener setOnSocketStatusListener)  
{  
    this.onSocketStatusListener = setOnSocketStatusListener;  
}
```

- 外部调用：用于设置 socket 连接服务器的地址。

```
public void setServerAddress(String serverAddress) {  
    this.serverAddress = serverAddress;  
}
```

- 外部调用：用于设置 socket 连接服务器的端口号。

```
public void setServerPort(int serverPort) {  
    this.serverPort = serverPort;  
}
```

1.3.4.3.4 socket 连接调用接口

```
/**  
 * socket连接  
 */  
public void connectSocket() {  
    new ConnectSocketThread().start();  
}
```

1.3.4.3.5 socket 发送数据调用接口

```
/**  
 * 发送数据给服务  
 *  
 * @param data byte数组  
 */  
public void senddataToserver(byte[] data) {  
    new SendDataToServerThread(data).start();  
}
```

1.3.4.3.6 socket 数据接收接口

```
/**
 * 从服务循环接收数据
 */
public void receiveLoopDataFromServer() {
    new ReceiveLoopDataFromServerThread().start();
}
```

1.3.4.3.7 关闭客户端 socket 接口

```
/**
 * 关闭客户端socket
 */
public void closeClientSocket() {
    new CloseClientSocketThread().start();
}
```

1.3.4.3.8 DNS 域名解析功能

示例代码路径:

WearTest\app\src\main\java\com\wrtm\weartest\componentsactivity\TransportationCodeActivity.java

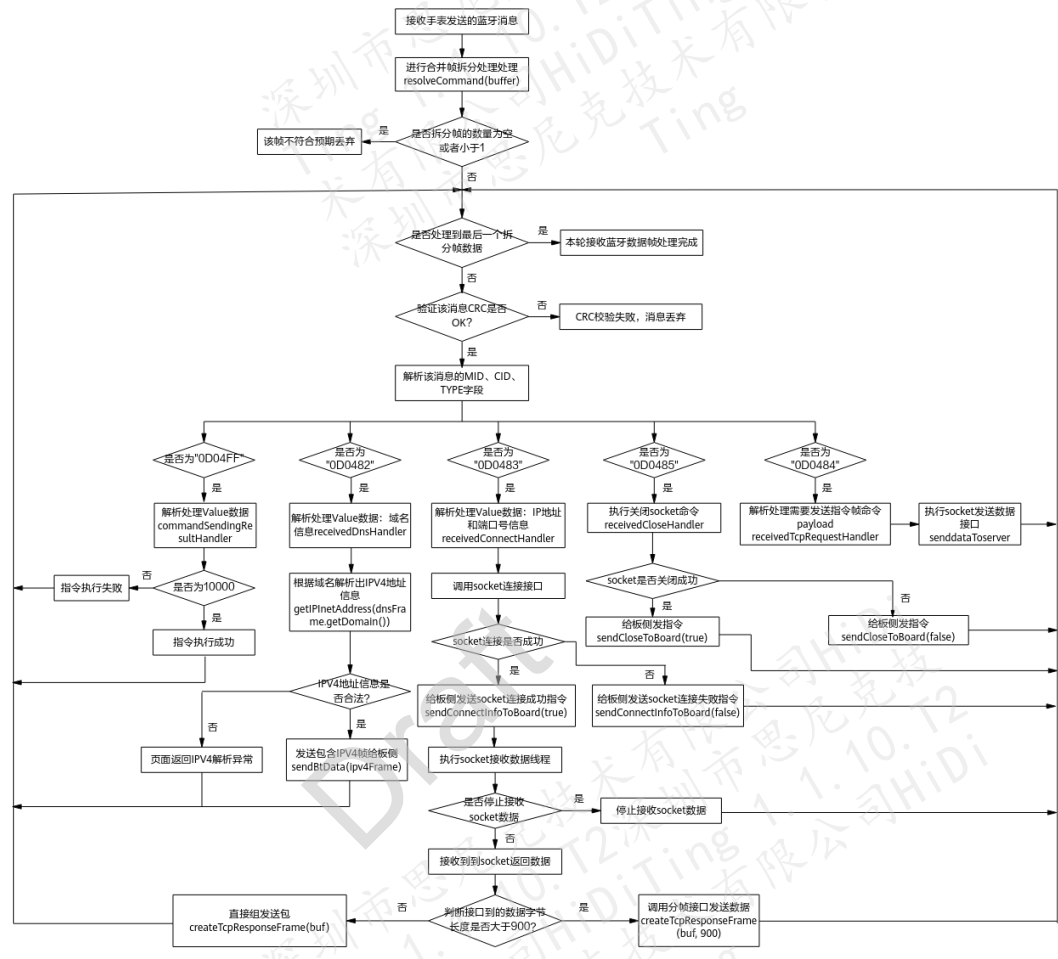
DNS 域名解析功能接口:

```
private void getIPInetAddress(String domain) {
    new GetIpThread(domain).start();
}
```

入参为域名，解析出域名的 IPv4 地址。

1.3.4.4 蓝牙指令&socket 交互示例代码讲解

图1-9 蓝牙 diag 指令和手机侧 socket 交互指令流程



该接口处理每次收到手表侧发来的蓝牙消息处理流程如下所示，参数 buffer 为蓝牙 SPP 通道接收到的数据帧，详细说明如下。

```
private void originResponseHandler(byte[] buffer) {
    try {
```

```
        //用于存储CRC校验通过符合规则的数据帧
```

```
        List<byte[]> availableCommandList;
```

```
        /*
```

蓝牙传输可能存在多个帧合并的情况，调用接口进行拆分帧，例如：

收到的帧为AF030800400D0481810179B5AF030B00400D048584000000008CF4

则需要拆分为AF030800400D0481810179B5和AF030B00400D048584000000008CF4

返回的列表则为2条数据帧: AF030800400D0481810179B5和

AF030B00400D048584000000008CF4

```

    */
    availableCommandList = CommandParse.resolveCommand(buffer);
    if (availableCommandList == null || availableCommandList.size() < 1) {
        return;
    }
    //遍历收到的数据帧，进行解析处理
    for (int i = 0; i < availableCommandList.size(); i++) {
        /*实例化CommandParse对象： CommandParse类主要用于解析数据帧结构，解析出
数据帧的crc字段、payload字段、mid字段、cid字段、type字段等。*/
        CommandParse recvValueCommandParse = new
        CommandParse(availableCommandList.get(i));
        //如果该数据帧CRC校验不通过，则不满足要求废弃
        if (!recvValueCommandParse.checkReceivedCrc16()) {
            return;
        }
        /*判断是否包含分帧字段，分帧参考1.3.4.4.5章节分帧讲解内容（APK收到板侧发送的
起始帧、过程帧以及完整帧，需要解析出完整帧的value值）*/
        if (recvValueCommandParse.isHasFrameStatus()){
            if (recvValueCommandParse.getCurrentFrameStatus() !=
            FrameStatus.FRAME_END) {
                return;
            }
            //收到终止帧，获取整个帧的value值
            Object obj = CommandParse.FramePayloadHashMap.get("VALUE_LIST");
            List<byte[]> valueList = Utils.convertList(obj);
            if (!valueList.isEmpty())
            {
                String MCAndType = (String)
                CommandParse.FramePayloadHashMap.get("MCT_ID");
                if (MCAndType.toUpperCase(Locale.ROOT).equals("0D0484") )
                {
                    byte[] requestvalue = valueList.get(0);
                    //获取到传输帧的value值后，给阿里服务器发送指令
                    receivedTcpRequestHandler(requestvalue);
                }
            }
            //将存储分帧的map清空
            CommandParse.clearFrameContainer();

```



```

        return;
    }

    //解析出蓝牙数据帧的Mid、Cid、Type字段

    switch
    (recvValueCommandParse.getMCAndTypeString().toUpperCase(Locale.ROOT)) {
        case "0D0482":
            //收到蓝牙域名解析相关帧，该接口主要用于解析域名，再把域名映射成IPV4地址、
            封装成蓝牙数据帧发送给手表

            receivedDnsHandler(recvValueCommandParse.getPayload());
            break;
        case "0D0483":
            //收到蓝牙socket连接相关帧，解析出服务器IP地址和端口号，调用socket连接接
            口

            receivedConnectHandler(recvValueCommandParse.getPayload());
            break;
        case "0D0485":
            //收到蓝牙关闭socket数据帧，调用closesocket接口关闭socket

            receivedCloseHandler(recvValueCommandParse.getPayload());
            break;
        case "0D0484":
            //收到手表给手机发送tcp上网请求的蓝牙数据指令，解析出来payload数据信息，
            组帧后调用发送socket指令接口

            receivedTcpRequestHandler(recvValueCommandParse.getPayload());
            break;
        case "0D04FF":
            //板侧返回命令执行成功失败指令

            commandSendingResultHandler(recvValueCommandParse.getPayload());
            break;
        default:
            break;
    }
}

} catch (IllegalArgumentException err) {
    CommonLog.debug(TAG, "An error occurred when parsing the return value." +
err.getMessage());
}
}

```

1.3.4.4.1 网络状态更新交互指令（手机->单板）

表1-7 手机给手表发送网络状态指令字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
Network_status	0x01	1	uint8	表示当前网络是否可用。 - value 值 0: 网络不可用。 - value 值 1: 网络可用。	M	1.0/1.0

- 请求帧结构:

Model ID 0xD	Command ID 0x4	Network_status TLV
-----------------	-------------------	-----------------------

- 应答帧结构:

- 返回成功:

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 100000
-----------------	-------------------	--------------	----------------	-----------------

- 返回失败:

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 1XXXXX
-----------------	-------------------	--------------	----------------	-----------------

蓝牙 SPP 发送网络请求报文示例（网络状态可用）:

AF 03 08 00 24 0D 04 81 81 01 C0 EC

表1-8 蓝牙 SPP 发送网络请求报文示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID	Command ID	Type	len	value	CRC
0xAF	0x03	0x0800	0x24	0x0D	0x04	0x81	0x81	0x01	0xC0EC

步骤 1 注册网络状态广播接收器。

手机侧注册网络状态广播接收器示例代码：

```
private void registerNetworkBroadcast() {  
    Log.e(TAG, "网络状态广播接收器");  
  
    netWorkReceiver = new NetworkReceiver();  
    IntentFilter filter = new IntentFilter();  
    filter.addAction(WifiManager.WIFI_STATE_CHANGED_ACTION);  
    filter.addAction(WifiManager.NETWORK_STATE_CHANGED_ACTION);  
    filter.addAction(ConnectivityManager.CONNECTIVITY_ACTION);  
    registerReceiver(netWorkReceiver, filter);  
}  
  
class NetworkReceiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        if (ConnectivityManager.CONNECTIVITY_ACTION.equals(intent.getAction())) {  
            NetworkInfo info =  
intent.getParcelableExtra(ConnectivityManager.EXTRA_NETWORK_INFO);  
            if (info == null) {  
                return;  
            }  
            if (info.getState() == NetworkInfo.State.CONNECTED && info.isAvailable()) {  
                if (info.getType() == ConnectivityManager.TYPE_WIFI  
                    || info.getType() == ConnectivityManager.TYPE_MOBILE) {  
                    Utils.sendMessage(EXECUTE_LOG, "网络已连接", handler, BLACK, 0);  
                    sendNetworkStatusFrame(true);  
                    return;  
                }  
            }  
            Utils.sendMessage(EXECUTE_LOG, "网络已断开", handler, RED, 0);  
            sendNetworkStatusFrame(false);  
        }  
    }  
}
```

步骤 2 蓝牙 SPP 通道发送网络状态消息给手表侧。

蓝牙 SPP 通道发送网络请求报文组帧示例代码：

```
private void sendNetworkStatusFrame(boolean isCanUse) {  
    byte[] networkUseFrame = CommandParse.createPhoneNetworkStatusFrame(isCanUse);
```

```

//发送网络状态可用帧示例: AF 03 08 00 24 0D 04 81 81 01 C0 EC
sendBtData(networkUseFrame);
}
/**
 * 手机给手表发送网络状态帧
 *
 * @param isCanUse true 网络可用false 网络不可用
 * @return byte[] 需要给手表发送的网络状态帧
 */
public static byte[] createPhoneNetworkStatusFrame(boolean isCanUse) {
    //获取网络请求帧value值
    byte[] networkStatusFrame = getPhoneNetworkStatusFrame(isCanUse);
    //获取MID、CID、TLV格式帧
    byte[] networkStatusFrameTlv = getMCAndTLV(
        CommonConstants.BT_RIDE_CODE_MID,
        CommonConstants.BT_RIDE_CODE_CID,
        CommonConstants.BT_NETWORK_STATUS_TYPE,
        networkStatusFrame
    );
    //添加CRC校验位
    byte[] tlvCrc = Utils.crc16(networkStatusFrameTlv);
    //返回完整的蓝牙SPP通道发送的数据帧
    return getUploadFrameCommand(
        CommonConstants.TRANS_FER_SOF,
        CommonConstants.TRANS_FER_CTRL,
        tlvCrc,
        networkStatusFrameTlv
    );
}

```

----结束

1.3.4.4.2 DNS 域名解析帧交互指令（单板->手机）

表1-9 域名解析字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
request_data	0x02	var	uint8[N]	域名信息	M	1.0/1.0
reponse_data	0x02	var	uint8[4]	IPV4 信息	M	1.0/1.0

- 请求帧结构：

Model ID 0xD	Command ID 0x4	request_data TLV
-----------------	-------------------	---------------------

- 应答帧结构：

Model ID 0xD	Command ID 0x4	reponse_data TLV
-----------------	-------------------	---------------------

蓝牙 SPP 域名解析请求帧结构示例（手机 APP→板侧的数据帧）：

AF 03 18 00 40 0D 04 82 91 69 6F 74 61 70 69 2E 61 6C 69 70 61 79 2E 63 6F 6D B8 3E

表1-10 蓝牙 SPP 域名解析请求帧结构示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID (1Byte)	Command ID (1Byte)	Type (1Byte)	len (1Byte)	Value (NByte)	CRC (2Byte)
0xAF	0x03	0x1800	0x40	0x0D	0x04	0x82	0x91	0x69 6F 74 61 70 69 2E 61 6C 69 70 61 79 2E 63 6F 6D	0xB83E

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID (1Byte)	Command ID (1Byte)	Type (1Byte)	len (1Byte)	Value (NByte)	CRC (2Byte)
								79 2E 63 6F 6D	

域名解析应答帧示例：

AF 03 0B 00 24 0D 04 82 84 CB D1 FA 12 61 69

表1-11 域名解析应答帧示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID (1Byte)	Command ID (1Byte)	Type (1Byte)	len (1Byte)	Value (NByte) IP 地址信息	CRC (2Byte)
0xAF	0x03	0x0B00	0x24	0x0D	0x04	0x82	0x84	0xCB D1FA 12	0x6169

示例代码逻辑如下，解析出域名信息然后转换成 IPV4 地址，组装成应答帧发送到手表端：

```
private void receivedDnsHandler(byte[] payload) {  
    //根据指令帧格式解析出DNS域名信息，例如上面域名解析请求帧结构示例中value字段：69 6F  
    74 61 70 69 2E 61 6C 69 70 61 79 2E 63 6F 6D  
    CommandParse.ParseDnsRequestFrame dnsFrame = new CommandParse  
        .ParseDnsRequestFrame(payload);  
    //解析出域名：696F746170692E616C697061792E636F6D-> iotapi.alipay.com  
    if (!dnsFrame.getDomain().isEmpty()) {  
        //通过域名获取IPV4地址信息  
        getIPInetAddress(dnsFrame.getDomain());  
    }  
}
```

相关使用接口如下：

```
/**
 * 通过域名解析出IPV4地址消息，封装成蓝牙数据帧发送给手表
 *
 * @param payload tlv数据
 * @return true / false
 */

private void getIPInetAddress(String domain)
{
    String ipv4Address = "";
    try {
        InetAddress[] inetAddresses = InetAddress.getAllByName(domain);
        for (InetAddress inetAddress : inetAddresses) {
            if (inetAddress instanceof Inet4Address) {
                ipv4Address = inetAddress.getHostAddress();
                break;
            }
        }
    } catch (UnknownHostException e) {
        e.printStackTrace();
    }
    if (ipv4Address == null || ipv4Address.isEmpty()) {
        Utils.sendMessage(EXECUTE_LOG, String.format(Locale.ROOT,
            "解析%s的ipv4地址错误，停止向手表发送ipv4信息", domain), handler, GREEN,
0);
        return;
    }
    Utils.sendMessage(EXECUTE_LOG, String.format(Locale.ROOT,
        "解析%s后的ipv4地址为: %s。即将向手表发送ip信息", domain, ipv4Address),
handler, BLACK, 0);

    //组合成手表IPV4帧
    byte[] ipv4Frame = CommandParse.createDnsResponseFrame(ipv4Address);
    /*手机侧将IPV4帧发送给手表，发送帧示例： AF 03 0B 00 24 0D 04 82 84 CB D1 FA 12 61
69*/
    sendBtData(ipv4Frame);
}

/**
 * 手机dns解析后的pv4信息帧
 */
```

```
* @param ipv4Info ipv4地址信息
* @return byte[]
*/
public static byte[] createDnsResponseFrame(String ipv4Info) {
    byte[] dnsResponseFrame = getDnsResponseFrame(ipv4Info);
    byte[] dnsResponseFrameTlv = getMCAndTLV(
        CommonConstants.BT_RIDE_CODE_MID,
        CommonConstants.BT_RIDE_CODE_CID,
        CommonConstants.BT_DNS_RESPONSE_TYPE,
        dnsResponseFrame
    );
    byte[] tlvCrc = Utils.crc16(dnsResponseFrameTlv);
    return getUploadFrameCommand(
        CommonConstants.TRANS_FER_SOF,
        CommonConstants.TRANS_FER_CTRL,
        tlvCrc,
        dnsResponseFrameTlv
    );
}
/**
 * 手机响应dns解析帧value结构
 *
 * @param ipv4Info ipv4信息
 * @return byte[]
 */
private static byte[] getDnsResponseFrame(String ipv4Info) {
    byte[] ipByteList = new byte[4];
    String[] ipSplitList = ipv4Info.split("\\.");
    for (int i = 0; i < ipSplitList.length; i++) {
        ipByteList[i] = (byte) Integer.parseInt(ipSplitList[i], 10);
    }
    return ipByteList;
}
```

1.3.4.4.3 建立 TCP 连接指令（单板->手机）

表1-12 建立连接字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
request_data	0x03	6	ip_info	Struct ip_info{ uint8 dst_ip[4] uint16 dsp_port } dst_ip: 目的地 址 dsp_port: 目的 端口	M	1.0/1.0
reponse_data	0x03	4	connect_result	Struct connect_result{ int32 ret int32 fd } ret: 0 连接成 功, 其他为错误 码。 ret 为 0 时, fd 为 TCP 连接句柄。	M	1.0/1.0

- 请求帧结构:

Model ID 0xD	Command ID 0x4	request_data TLV
-----------------	-------------------	---------------------

- 应答帧结构:

Model ID 0xD	Command ID 0x4	reponse_data TLV
-----------------	-------------------	---------------------

蓝牙 SPP 建立连接请求帧结构示例 (板侧→手机 APP 的数据帧):

AF 03 0D 00 40 0D 04 83 86 CB D1 FA 12 BB 01 3B 87

表1-13 蓝牙 SPP 建立连接请求帧结构示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID (1Byte)	Command ID (1Byte)	Type (1Byte)	len (1Byte)	Value (NByte)	CRC (2Byte)
0xAF	0x03	0x0D00	0x40	0x0D	0x04	0x83	0x86	0xCB D1 FA 12 BB 01	0x3B 87

建立连接响应帧结构示例（手机 APP→板侧数据帧）：

AF 03 0F 00 24 0D 04 83 88 00 00 00 00 01 00 00 00 89 D9

表1-14 建立连接响应帧结构示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID (1Byte)	Command ID (1Byte)	Type (1Byte)	len (1Byte)	Value (NByte)	CRC (2Byte)
0xAF	0x03	0x0F00	0x24	0x0D	0x04	0x83	0x88	0x00 0000 00 01 0000 00	0x89D 9

示例代码逻辑如下：

步骤 1 根据指令帧格式解析出服务器 ip 地址和端口信息，进行 socket 连接。

```
private void receivedConnectHandler(byte[] payload) {  
    //根据指令帧格式解析出服务器ip地址和端口信息CB D1 FA 12 BB 01  
    CommandParse.ParseConnectRequestFrame connectFrame = new CommandParse  
        .ParseConnectRequestFrame(payload);  
    /*给socket对象设置IP和端口号、进行socket连接，解析出IP和PORT：CB D1 FA 12 -  
>203.209.250.18（4个字节分别转成十进制IP地址数字，再用"."拼接成IP地址）
```

```
BB 01 -> :443 (2个字节解析出端口号为443) */
```

```
createSocketClient(connectFrame.getDstIp(), connectFrame.getDstPort());
```

```
//调用socket连接接口执行连接阿里服务器功能
```

```
socketClient.connectSocket();
```

```
}
```

步骤 2 socket 连接后手机侧需要给板子发送连接状态指令，成功参数传为 true、失败为 false。

```
private void sendConnectInfoToBoard(boolean isConnected) {
```

```
    byte[] connectFrame = CommandParse.createSocketConnectFrame(isConnected ? 0 : -1,
```

```
        isConnected ? 1 : 0);
```

```
    /*完成socket连接响应帧组帧，组帧示例：AF 03 0F 00 24 0D 04 83 88 00 00 00 00 01 00 00
```

```
00 89 D9*/
```

```
    sendBtData(connectFrame);
```

```
}
```

----结束

步骤 2 中使用到的相关接口如下：

```
/**
```

```
 * 创建socket连接帧状态帧
```

```
 *
```

```
 * @param ret 连接结果
```

```
 * @param fd socket句柄
```

```
 * @return byte[]
```

```
 */
```

```
public static byte[] createSocketConnectFrame(int ret, int fd) {
```

```
    byte[] connectResultFrame = getConnectResultFrame(ret, fd);
```

```
    byte[] connectResultFrameTlv = getMCAndTLV(
```

```
        CommonConstants.BT_RIDE_CODE_MID,
```

```
        CommonConstants.BT_RIDE_CODE_CID,
```

```
        CommonConstants.BT_SOCKET_CONNECT_RESPONSE_TYPE,
```

```
        connectResultFrame
```

```
    );
```

```
    byte[] tlvCrc = Utils.crc16(connectResultFrameTlv);
```

```
    return getUploadFrameCommand(
```

```
        CommonConstants.TRANS_FER_SOF,
```

```
        CommonConstants.TRANS_FER_CTRL,
```

```
        tlvCrc,
```

```
        connectResultFrameTlv
```

```
);  
}
```

1.3.4.4.4 socket 数据读取

“1.3.4.4.3 建立 TCP 连接指令（单板->手机）” 章节 socket 连接成功后即可调用 socket 接收数据线程接口，调用 socket 接收数据线程示例代码如下所示：

```
private class ReceiveLoopDataFromServerThread extends Thread {  
    /**  
     * 构造方法  
     */  
    public ReceiveLoopDataFromServerThread() {  
        super.setName("ReceiveDataFromServerThread");  
    }  
    @Override  
    public void run() {  
        try {  
            doReceiveTask();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
    private void doReceiveTask() throws IOException {  
        ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();  
        while (isLoopReveiveData) {  
            if (clientSocket == null) {  
                onSocketStatusListener.socketClientRecieveFailed("socket为null，接受异常");  
                return;  
            }  
            if (!clientSocket.isConnected()) {  
                onSocketStatusListener.socketClientRecieveFailed("socket没有连接，接受异常");  
                return;  
            }  
            InputStream inFromServer = clientSocket.getInputStream();  
            DataInputStream in = new DataInputStream(inFromServer);  
            byte[] buffer = new byte[1024];  
            int len;  
            byteArrayOutputStream.reset();  
            while ((len = in.read(buffer)) != -1) {  
                byteArrayOutputStream.write(buffer, 0, len);  
                boolean hasDataFlag = false;
```

```
long startTime = System.currentTimeMillis();
//由于网络传输可能存在时延，本demo APK按照在30ms之内收到的数据合并为一
包处理

while (System.currentTimeMillis() - startTime < 30) {
    if (in.available() != 0) {
        CommonLog.debug("doReceiveTask", "in.available() != 0");
        hasDataFlag = true;
        break;
    }
    SystemClock.sleep(1);
}
if (hasDataFlag) {
    continue;
}
byte[] completeResponse = byteArrayOutputStream.toByteArray();
/*socket读取数据完成后，将读到的数据传入回调函数，此回调函数的处理逻辑会
在“1.3.4.4.5 TCP响应数据指令”章节讲解*/

onSocketStatusListener.socketClientRecieveSuccess(completeResponse);
byteArrayOutputStream.reset();
break;
}
}
}
```

1.3.4.4.5 TCP 响应数据指令（手机->单板）

socket 数据读取完成后手机侧需要把数据封装为 TCP 响应数据帧发给手表

表1-15 服务器 TCP 响应数据字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
reponse_data	0x04	var	uint8[N]	此处为 TCP 的响应报文数据	M	1.0/1.0

- 请求帧结构：

Model ID 0xD	Command ID 0x4	reponse_data TLV
-----------------	-------------------	---------------------

● 应答帧结构：

- 返回成功：

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 100000
-----------------	-------------------	--------------	----------------	-----------------

- 返回失败：

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 1XXXXX
-----------------	-------------------	--------------	----------------	-----------------

socket 读取数据完成后，回调函数处理逻辑如下：

须知

按照多少字节分帧根据蓝牙通道传输能力设定、例如可以设置按照 1024 分帧或者 2048 分帧，本文中 demo 用例按照 900 字节进行分帧。

```
@Override
public void socketClientRecieveSuccess(byte[] buf) {
    /*buf为socket服务器端接收到的字节数据，本接口功能为把数据封装为TCP响应帧发送给手表*/
    if (buf.length <= 900) {
        //如果socket接收字节长度小于等于900，则执行不分帧组包流程

        byte[] tcpResponseFrame = CommandParse.createTcpResponseFrame(buf);
        sendBtData(tcpResponseFrame);
    } else {
        /*如果socket接受字节长度大于900，等进行分帧*/

        List<byte[]> responseFrameList = CommandParse.createTcpResponseFrame(buf, 900);
        for (byte[] item : responseFrameList) {
            sendBtData(item);
        }
    }
    Utils.sendMessage(EXECUTE_LOG, String.format(Locale.ROOT,
        "转发tcp响应到手表侧成功：已转发%d bytes", buf.length), handler, GREEN, 0);
}
```

分帧的数据帧格式说明请参见《穿戴类产品对外交互协议》的“链路层通用帧格式定义-控制域 CTRL” 章节说明。

蓝牙 SPP 回复服务器 TCP 响应不需要分帧处理的数据逻辑

蓝牙 SPP 回复服务器 TCP 响应不需要分帧报文示例（手机 APP→板侧的数据帧）：

```
AF 03 02 02 24 0D 04 84 02 F9 01 17 03 03 01 F4 FB 2E 10 06 89 2F 00 76 80 6A
68 5A 75 D2 A8 AD 88 58 43 A0 06 AA E6 A5 F1 D9 C9 54 80 70 00 BF A3 C1 98 C4
7B B9 EC 16 94 3F D8 89 6C 96 57 F6 8D AE C8 97 D2 43 0B 9A 7E 58 05 E1 EC
58 BE 4F C2 7F AD 90 20 9E 08 22 DF A4 C6 BB 25 F2 EE 32 0D BB 52 3D A2 8B
95 8D B8 1B 42 42 4A 4A B2 59 61 12 C3 D0 0B F2 E3 BC 2B 0E 6E D5 7E 14 55
B4 DC 77 7E D8 36 8A 62 2A F3 8D 74 FF 06 EF 45 7B 10 00 0C F0 FC 0D 52 66
19 F6 3E 71 BF 57 FD 3B 56 90 1B C6 93 5D 31 28 1E 93 BB 32 0E 12 B9 47 4B B0
1E FF 22 A7 67 DC 60 88 38 8B 06 F9 2B B3 F8 13 08 98 DE 82 ED DA 6D FAA3
45 92 96 3E 5B 7B 2D 63 E2 C5 86 28 34 3F B8 60 E7 B1 A1 F6 F8 D4 05 AF B4 9D
56 C1 13 81 64 BB 2C 85 57 13 7F 15 BC 5B 04 EB DE 73 B8 31 E8 22 DE 81 EF
8E 09 02 A2 E0 D2 72 43 11 B0 77 2B 65 D0 91 3D 67 BD D1 E9 17 8A B7 A0 A8 A9
FB F4 4F CC 8E A3 EB BC BA 58 4D EC B5 C1 4C 00 94 29 5C 4A 44 26 3A D9 69
16 EE 08 D8 E9 77 C1 54 93 6D F7 3C 20 2E 36 02 12 E9 5D 52 0D 69 0A 8A 3E
5C 33 B6 8E 53 DD 2F 4A DD 94 43 07 D3 01 68 2C E5 65 A8 4C EF D3 44 2B E3
48 44 6A 8C 3E 3F FF 29 4B D0 41 80 B7 0B 73 8D 3A C3 D6 E0 0A 6F 3F F6 05
D8 67 B1 07 EC 2B 46 34 B7 9D 6C 82 2E 20 EC C9 4F 3B EA F0 87 9F 6D D8 29
8A 49 1E DF 39 73 69 53 C1 F0 D8 B9 38 03 03 18 51 53 09 C0 8A 61 5B A2 9C 9D
2B E1 84 2D B4 B0 01 F4 3A 49 4D 30 A2 8D 30 69 B5 5B 75 2C 61 A8 F1 C9 B7
FA BA FE CB 1F 16 CD 59 20 69 16 05 87 77 78 3D C1 F2 12 55 6A 3A 63 FE 57 D7
DA FF 08 D4 34 76 FD ED EA FC EA 77 F1 AF EF 21 08 1F 5E 6E 95 00 F6 C2 66
40 54 C5 12 86 43 E8 91 D0 4F EB
```

表1-16 蓝牙 SPP 回复服务器 TCP 响应不需要分帧报文示例

SO F (1 Byte)	C T R L (1 Byte)	LE N (3 Byte)	FID (1Byte)	M o d e l ID (1 Byte)	C o m m a n d ID (1 Byte)	Typ e (1Byte)	len (3Byte)	Value (NByte) IP 地址信息	CRC (2Byte)
0x AF	0x 03	0x 02 02 24	0x2 4	0x 0 D	0x 04	0x8 4	0x02 F901	0x17030301F4FB2E1 006892F0076806A68 5A75D2A8AD885843 A006AAE6A5F1D9C9 54807000BFA3C198C 47BB9EC16943FD88 96C9657F68DAEC89 7D2430B9A7E5805E1 EC58BE4FC27FAD90 209E0822DFA4C6BB 25F2EE320DBB523D A28B958DB81B42424	0x4FE B

SO F (1 Byte)	C T R L (1 Byte)	LE N (3 Byte)	FID (1B yte)	M od el ID (1 B yte)	C o m m a n d ID (1 B yte)	Typ e (1B yte)	len (3Byte)	Value (NByte) IP 地 址信息	CRC (2Byt e)
								A4AB2596112C3D00 BF2E3BC2B0E6ED57 E1455B4DC777ED83 68A622AF38D74FF06 EF457B10000CF0FC 0D526619F63E71BF5 7FD3B56901BC6935 D31281E93BB320E12 B9474BB01EFF22A76 7DC6088388B06F92B B3F8130898DE82ED DA6DFAA34592963E 5B7B2D63E2C586283 43FB860E7B1A1F6F8 D405AFB49D56C113 8164BB2C8557137F1 5BC5B04EBDE73B83 1E822DE81EF8E0902 A2E0D2724311B0772 B65D0913D67BDD1E 9178AB7A0A8A9FBF 44FCC8EA3EBBCBA 584DECB5C14C0094 295C4A44263AD9691 6EE08D8E977C15493 6DF73C202E360212E 95D520D690A8A3E5 C33B68E53DD2F4AD D944307D301682CE5 65A84CEFD3442BE3 48446A8C3E3FFF294 BD04180B70B738D3 AC3D6E00A6F3FF60 5D867B107EC2B4634 B79D6C822E20ECC9 4F3BEAF0879F6DD8 298A491EDF3973695 3C1F0D8B938030318 515309C08A615BA29 C9D2BE1842DB4B00 1F43A494D30A28D30	

SOF (1 Byte)	CTRL (1 Byte)	LEN (3 Byte)	FID (1Byte)	Model ID (1 Byte)	Command ID (1 Byte)	Type (1Byte)	len (3Byte)	Value (NByte) IP 地址信息	CRC (2Byte)
								69B55B752C61A8F1 C9B7FABAFECB1F16 CD592069160587777 83DC1F212556A3A63 FE57D7DAFF08D434 76FDEDEAFCEA77F1 AFEF21081F5E6E950 0F6C2664054C51286 43E891D0	

不分帧示例代码逻辑如下：

```
/**
 * 阿里socket服务端返回不需要分帧处理的数据
 *
 * @param responseData 原始数据
 * @return 字节数组
 */
public static byte[] createTcpResponseFrame(byte[] responseData) {
    //responseData为socket收到的服务器端的数据，getMCAndTLV接口用于组成数据帧的MID、
    CID、TLV 字节数据

    byte[] tcpResponseFrameTlv = getMCAndTLV(
        CommonConstants.BT_RIDE_CODE_MID,
        CommonConstants.BT_RIDE_CODE_CID,
        CommonConstants.BT_SOCKET_TCP_RESPONSE_TYPE,
        responseData
    );

    //获取CRC数据

    byte[] tlvCrc = Utils.crc16(tcpResponseFrameTlv);

    //最终添加SOF、CTRL字段组成完整的数据帧格式，参考上述表2数据
```

```
return getUploadFrameCommand(  
    CommonConstants.TRANS_FER_SOF,  
    CommonConstants.TRANS_FER_CTRL,  
    tlvCrc,  
    tcpResponseFrameTlv  
);  
}
```

蓝牙 SPP 回复服务器 TCP 响应需要分帧的处理逻辑

阿里服务器返回的数据存在超过 4KB 的场景，针对这种场景需要进行分帧处理，分帧相关的帧格式请参考《穿戴类产品对外交互协议》的“链路层通用帧格式定义-控制域 CTRL”章节说明。

分帧原理如下（本小结按照 500 字节分帧举例，实际可以根据蓝牙传输能力大小按照字节分帧）：

说明

黄色标识分为起始帧、绿色和蓝色分别为过程帧、紫色为结束帧。

1. 收到阿里服务器返回的数据报文：

170303066E268355074CCC17E21CA33FDDC85D7F02897E33F803758F2F07
384DABA3B128526343868712BB58ECDC9369AC621510CC33756094CDDDB
44FF81E28BB1F12AC974A5E7553CBCADFDB69504004FCB0F6CBB713D3B
C43AFB961E753420E0A34FFF3FF4198CB5DB8A714AC10FC93278AFDA74D
86C6B7687F0880856903152A206D743D2E8C8B89CDE2EE9157A351040E30
42920EF86CC5CD978410587CF3932A323BB99F1C4BE6F7116003889EAA
E54E43ADC1A417B94E230F494E1BE71042F398853248FC2C0578BDF94C2
BBF09265F8120BCE70EE3B4ABDD9B08C23415013EBBD244991862E761BC
6F936AC17B99377A3564ACBCD791B276F074875B94FDC4DF3F5C73F40C1
24DC0C68E034260B71D445CE80CAC4EB67C5EB6A2DCA92538722C45AF6
52D5A676C8A7867DDC8C7614A3D516C0D49359948B8915FB6588B5B22170
2ECAB4C65B083DC399E7BBDA8F77F68165B9D4E922495C7B1F6FFA67ED
0CE192F893ADE5F8C08FC6D7E62199102D8674C97003A2DCACD29674F27
82082D7B9FF4152BDAFBFDBC9B52D42A05286C930FF4AAED9E037FAFE8
2D13439EA2DD1D8855038D7373B0E517EB71BC0B34A3FB449EFAA7C21E8
AB2F2420C8BDC398157B670B3EC8167C3FFA07ADB5FA31034AD2C5E1DD
93E8E6825BC3DA3276A92E59C6901AA9197C0C851737752404654BF7A940
E0706B7F6CF752CA11DFAB3A9FCD949A095F0EC872D37157CE2D04C3A65
9C21306EAA4810925B2D0E9455BC160735876B3ABB356B5B236D77F4896E
947BD067CEFFFCF4DE7DCABA5152B8C0870F5284C590241B6B1062D4414
7CF1C77081A4C233F529DDEAA51C07700817145058928C77938BD3D4ECE
07467BB961BFEF465967FE45ACDF11C14BCCD491ECE3421DB8CBBE9EA4
A4AE1329FC33FA227BB37A927BFDE6BEE93501C046D7F0489A79173C3C9
95A35C85AC81D9A27357D8DD193A7556DC89E42BEF51A3DD6561A30DCD
E2CFAB982A8F24B0E5677A7B64EE0DAA4A60F83E1FB8521902F8D8F4DC9
63508011D73B2B7647A97E0056C8670539E3FA788D6594C07838D0F34F15A
18AEBF5EECA0C956BEC5811D25F1B82E7DA9DFA00B52FBBF0BEF82984
4144A7DB0B5FFDD31725121B76CA5171BAD6A41A158EB50B93A48F47DBE

D738C3DEDC229612B57E7506DD7F0B8029E0D0C2B2DBBA2DB4FB056023
 1BC72542702A27395F97FF3E8FAE54BFB2488A2863783FA0A4D0FEDE5F34
 9673BADAE7FB2DE222F0D58634E173D5FC9DFFF7CEEC179DD941A74BD
 D09A9209C7AF38F8565778261ACEEDCCE1BFEE4BCCA03D2BFD04EFDB5
 D082C70E06875C673C62DAB179FB96F83AB7001A94DDB932DF6DA6D4375
 5882278CBCDB5E09BF7FD550AA217579EFB1FED11B0FA5256E9442F9E7A
 542357EE31C3F7F295AB2540BFCF6C2D636CAB932D1333EE915702CF39C
 6449E861A848ACD340CCFEAB54C97FD63F350CB32852E771A3A2F523489
 5A6B95C7E8F1E4095288595A802255FE27B27B68312733DFACDDBBAF8D4
 6C879449DF5D28EDC5E857878280CB25B2E890E151CE83B0A7EB21B60DA
 7EA870E7499B396C931F6EB413352CA5D65D2B5574A67A01A391C113928F
 F3E9F685D6BA1FC58D8860724D208BA03A68E8FA067D434FF740AD8564D
 9F9924B316D6456FBB892A2200A63B67B7D2CD87665559E1B5AA562CD6F
 A91D7FA3F75CBBABF84475CF059327DBFF55BEE674FA55ECE5C3920F9400
 F61DE7967632181D8C3DCDB519745D9847318466015B71D74ACA77FE0FA5
 A5A6777623ECF6CFE4EDEF4F5F297F6A38180DA87E08200981091370E39C
 5477653C4635183FBE0E02DA3F158E85FD984B6A6BC6E6B7A9156BF62A9
 A12477A6BC9403412E6FF3EA310F163E3E46898D0F22BF4B26E32391FAE1
 B79B8912794ECB8C28510EBB7BFB06470860D9E20288150F50C0C8E20184
 CA26026BE1583802111266F1AD3484B33CD8445A395E13D850ADF71FB5F8
 50512EF4872D8D4F0BD3BF768B968412F41609F4D89B3B6338C891AA3B69
 8B235EC0E6FACAE7EDD595376D9BEDBDD13751B816C1E6D7E25A83B051
 9214053FBE3A82B33AE4C4EC1C34F4C263A9D98ED69E73EA7E2906F5677
 6E910964A4A463E640D803E0B9DCE0CCB88C41079A662198EB905FD8745
 263C7B2A322996AEA7975D453D1A273E5476AD829D411D025684C1B2C28B
 5022766C9091CDCE438A99ABF1D9D4C6E3761B11EC3E11ABD396C188768
 9A0B070D5EB078D1A643959727647AA7F58A75A449389AAC1DCFCB64262
 9004E0DC645C048EFD65E7CE32FC1591CE67AB2F8496411A8B2FE1

2. 按照以下格式组成完整帧。

Model ID 0xD	Command ID 0x4	reponse_data TLV
-----------------	-------------------	---------------------

表1-17 TCP 响应数据帧字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
reponse_data	0x04	var	uint8[N]	此处为 TCP 的响 应报文数据	M	1.0/1.0

0D 为 ModelID、04 为 Command ID、84 是 Type、027306 为 TLV 里的 Length，
 其余从 1703 字节开始为实际的 value 值：

0D0484027306170303066E268355074CCC17E21CA33FDDC85D7F02897E33
 F803758F2F07384DABA3B128526343868712BB58ECDC9369AC621510CC33
 756094CDDDB44FF81E28BB1F12AC974A5E7553CBCADFDB69504004FCB0
 F6CBB713D3BC43AFB961E753420E0A34FFF3FF4198CB5DB8A714AC10FC9

3278AFDA74D86C6B7687F0880856903152A206D743D2E8C8B89CDE2EE91
57A351040E3042920EF86CC5CD978410587CF3932A323BB99F1C4BE6F711
6003889EAACE54E43ADC1A417B94E230F494E1BE71042F398853248FC2C0
578BDBFA94C2BBF09265F8120BCE70EE3B4ABDD9B08C23415013EBBD2449
91862E761BC6F936AC17B99377A3564ACBCD791B276F074875B94FDC4DF
3F5C73F40C124DC0C68E034260B71D445CE80CAC4EB67C5EB6A2DCA925
38722C45AF652D5A676C8A7867DDC8C7614A3D516C0D49359948B8915FB
6588B5B221702ECAB4C65B083DC399E7BBDA8F77F68165B9D4E922495C7
B1F6FFA67ED0CE192F893ADE5F8C08FC6D7E62199102D8674C97003A2DC
ACD29674F2782082D7B9FF4152BDAFBFDBC9B52D42A05286C930FF4AAE
D9E037FAFE82D13439EA2DD1D8855038D7373B0E517EB71BC0B34A3FB44
9EFAA7C21E8AB2F2420C8BDC398157B670B3EC8167C3FFA07ADB5FA3103
4AD2C5E1DD93E8E6825BC3DA3276A92E59C6901AA9197C0C85173775240
4654BF7A940E0706B7F6CF752CA11DFAB3A9FCD949A095F0EC872D37157
CE2D04C3A659C21306EAA4810925B2D0E9455BC160735876B3ABB356B5B
236D77F4896E947BD067CEFFFCF4DE7DCABA5152B8C0870F5284C590241
B6B1062D44147CF1C77081A4C233F529DDEAA51C07700817145058928C77
938BD3D4ECE07467BB961BFEF465967FE45ACDF11C14BCCD491ECE3421
DB8CBBE9EA4A4AE1329FC33FA227BB37A927BFDE6BEE93501C046D7F04
89A79173C3C995A35C85AC81D9A27357D8DD193A7556DC89E42BEF51A3D
D6561A30DCDE2CFAB982A8F24B0E5677A7B64EE0DAA4A60F83E1FB85219
02F8D8F4DC963508011D73B2B7647A97E0056C8670539E3FA788D6594C07
838D0F34F15A18AEABF5EECA0C956BEC5811D25F1B82E7DA9DFA00B52F
BBF0BEF829844144A7DB0B5FFDD31725121B76CA5171BAD6A41A158EB50
B93A48F47DBED738C3DEDC229612B57E7506DD7F0B8029E0D0C2B2DBBA
2DB4FB0560231BC72542702A27395F97FF3E8FAE54BFB2488A2863783FA0
A4D0FEDE5F349673BADAEE7FB2DE222F0D58634E173D5FC9DFFF7CEEC17
9DD941A74BDD09A9209C7AF38F8565778261ACEEDCCE1BFEE4BCCA03D
2BFD04EFDB5D082C70E06875C673C62DAB179FB96F83AB7001A94DDB932
DF6DA6D43755882278CBCEB5E09BF7FD550AA217579EFB1FED11B0FA525
6E9442F9E7A542357EE31C3F7F295AB2540BFCF6C2D636CAB932D1333EE
915702CF39C6449E861A848ACD340CCFEAB54C97FD63F350CB32852E771
A3A2F5234895A6B95C7E8F1E4095288595A802255FE27B27B68312733DFA
CDDBBABF8D46C879449DF5D28EDC5E857878280CB25B2E890E151CE83B0
A7EB21B60DA7EA870E7499B396C931F6EB413352CA5D65D2B5574A67A01
A391C113928FF3E9F685D6BA1FC58D8860724D208BA03A68E8FA067D434F
F740AD8564D9F9924B316D6456FBB892A2200A63B67B7D2CD87665559E1B
5AA562CD6FA91D7FA3F75CBBAF84475CF059327DBFF55BEE674FA55ECE
5C3920F9400F61DE7967632181D8C3DCDB519745D9847318466015B71D74
ACA77FE0FA5A5A6777623ECF6CFE4EDEF4F5F297F6A38180DA87E082009
81091370E39C5477653C4635183FBE0E02DA3F158E85FD984B6A6BC6E6B7
A9156BF62A9A12477A6BC9403412E6FF3EA310F163E3E46898D0F22BF4B2
6E32391FAE1B79B8912794ECB8C28510EBB7BFB06470860D9E20288150F5
0C0C8E20184CA26026BE1583802111266F1AD3484B33CD8445A395E13D85
0ADF71FB5F850512EF4872D8D4F0BD3BF768B968412F41609F4D89B3B633
8C891AA3B698B235EC0E6FACAE7EDD595376D9BEDBDD13751B816C1E6D
7E25A83B0519214053FBE3A82B33AE4C4EC1C34F4C263A9D98ED69E73EA
7E2906F56776E910964A4A463E640D803E0B9DCE0CCB88C41079A662198E
B905FD8745263C7B2A322996AEA7975D453D1A273E5476AD829D411D0256
84C1B2C28B5022766C9091CDCE438A99ABF1D9D4C6E3761B11EC3E11AB
D396C1887689A0B070D5EB078D1A643959727647AA7F58A75A449389AAC1
DCFCB642629004E0DC645C048EFD65E7CE32FC1591CE67AB2F8496411A8
B2FE1

3. 以上数据按照《穿戴类产品对外交互协议》可分为起始帧、过程帧、结束帧：

- a. 起始帧，红色表示标识为第 0 个帧（手机 APP→板侧的数据帧）：

```
AF07F8012400D0484027306170303066E268355074CCC17E21CA33FD
DC85D7F02897E33F803758F2F07384DABA3B128526343868712BB58EC
DC9369AC621510CC33756094CDDDB44FF81E28BB1F12AC974A5E755
3CBCADFDDB69504004FCB0F6CBB713D3BC43AFB961E753420E0A34FF
F3FF4198CB5DB8A714AC10FC93278AFDA74D86C6B7687F0880856903
152A206D743D2E8C8B89CDE2EE9157A351040E3042920EF86CC5CD97
8410587CF3932A323BB99F1C4BE6F7116003889EAACE54E43ADC1A41
7B94E230F494E1BE71042F398853248FC2C0578BDF9A9C2BBF09265F8
120BCE70EE3B4ABDD9B08C23415013EBBD244991862E761BC6F936A
C17B99377A3564ACBCD791B276F074875B94FDC4DF3F5C73F40C124D
C0C68E034260B71D445CE80CAC4EB67C5EB6A2DCA92538722C45AF6
52D5A676C8A7867DDC8C7614A3D516C0D49359948B8915FB6588B5B2
21702ECAB4C65B083DC399E7BBDA8F77F68165B9D4E922495C7B1F6F
FA67ED0CE192F893ADE5F8C08FC6D7E62199102D8674C97003A2DCA
CD29674F2782082D7B9FF4152BDAFBFDBC9B52D42A05286C930FF4A
AED9E037FAFE82D13439EA2DD1D8855038D7373B0E517EB71BC0B34
A3FB449EFAA7C21E8AB2F2420C8BDC398157B670B3EC8167C3FFA07
ADB5FA31034AD2C5E1DD93E8E6825BC3DA3276A92E59C6901AA9197
C0C85173778DEA
```

- b. 过程帧 1，红色表示标识为第 1 个帧（手机 APP→板侧的数据帧）：

```
AF0BF801240152404654BF7A940E0706B7F6CF752CA11DFAB3A9FCD9
49A095F0EC872D37157CE2D04C3A659C21306EAA4810925B2D0E9455
BC160735876B3ABB356B5B236D77F4896E947BD067CEFFFCF4DE7DC
ABA5152B8C0870F5284C590241B6B1062D44147CF1C77081A4C233F52
9DDEAA51C07700817145058928C77938BD3D4ECE07467BB961BFEF46
5967FE45ACDF11C14BCCD491ECE3421DB8CBBE9EA4A4AE1329FC33
FA227BB37A927BFDE6BEE93501C046D7F0489A79173C3C995A35C85A
C81D9A27357D8DD193A7556DC89E42BEF51A3DD6561A30DCDE2CFA
B982A8F24B0E5677A7B64EE0DAA4A60F83E1FB8521902F8D8F4DC963
508011D73B2B7647A97E0056C8670539E3FA788D6594C07838D0F34F1
5A18AEABF5EECA0C956BEC5811D25F1B82E7DA9DFA00B52FBBF0BE
F829844144A7DB0B5FFDD31725121B76CA5171BAD6A41A158EB50B93
A48F47DBED738C3DEDC229612B57E7506DD7F0B8029E0D0C2B2DBB
A2DB4FB0560231BC72542702A27395F97FF3E8FAE54BFB2488A286378
3FA0A4D0FEDE5F349673BADAE7FB2DE222F0D58634E173D5FC9DFFF
7CEEC179DD941A74BDD09A9209C7AF38F8565778261ACEEDCCE1BF
EE4BCCA03D2BFD04EFDB5D082C70E06875C673C62DAB179FB96F83
AB7001A94DDB932DF6DA6D43755882278CBCDB5E09BF7FD550AA217
579EFB1FED11B0FA9AEA
```

- c. 过程帧 2，红色表示标识为第 2 个帧（手机 APP→板侧的数据帧）：

```
AF0BF80124025256E9442F9E7A542357EE31C3F7F295AB2540BFCF6C2
D636CAB932D1333EE915702CF39C6449E861A848ACD340CCFEAB54C
97FD63F350CB32852E771A3A2F5234895A6B95C7E8F1E4095288595A8
02255FE27B27B68312733DFACDDBBAF8D46C879449DF5D28EDC5E85
7878280CB25B2E890E151CE83B0A7EB21B60DA7EA870E7499B396C93
1F6EB413352CA5D65D2B5574A67A01A391C113928FF3E9F685D6BA1F
C58D8860724D208BA03A68E8FA067D434FF740AD8564D9F9924B316D
6456FBB892A2200A63B67B7D2CD87665559E1B5AA562CD6FA91D7FA3
F75CBBAF84475CF059327DBFF55BEE674FA55ECE5C3920F9400F61DE
7967632181D8C3DCDB519745D9847318466015B71D74ACA77FE0FA5A
5A6777623ECF6CFE4EDEF4F5F297F6A38180DA87E08200981091370E3
```

9C5477653C4635183FBE0E02DA3F158E85FD984B6A6BC6E6B7A9156B
F62A9A12477A6BC9403412E6FF3EA310F163E3E46898D0F22BF4B26E3
2391FAE1B79B8912794ECB8C28510EBB7BFB06470860D9E20288150F5
0C0C8E20184CA26026BE1583802111266F1AD3484B33CD8445A395E13
D850ADF71FB5F850512EF4872D8D4F0BD3BF768B968412F41609F4D8
9B3B6338C891AA3B698B235EC0E6FACAE7EDD595376D9BEDBDD1375
1B816C1E6D7E25A83B0519214053FBE3A82B33AE4C4EC1C34F4C263A
9087C

- d. 结束帧，红色表示标识为第 3 个帧（手机 APP→板侧的数据帧）：

AF0FA1002403D98ED69E73EA7E2906F56776E910964A4A463E640D803
E0B9DCE0CCB88C41079A662198EB905FD8745263C7B2A322996AEA79
75D453D1A273E5476AD829D411D025684C1B2C28B5022766C9091CDC
E438A99ABF1D9D4C6E3761B11EC3E11ABD396C1887689A0B070D5EB
078D1A643959727647AA7F58A75A449389AAC1DCFCB642629004E0DC
645C048EFD65E7CE32FC1591CE67AB2F8496411A8B2FE10BCB

【蓝牙 SPP 起始报文帧】

蓝牙 SPP 起始报文示例（手机 APP→板侧的数据帧）：

AF07F8012400D0484027306170303066E268355074CCC17E21CA33FDDC85D7F
02897E33F803758F2F07384DABA3B128526343868712BB58ECDC9369AC621510
CC33756094CDDDB44FF81E28BB1F12AC974A5E7553CBCADFDDB69504004FCB
0F6CBB713D3BC43AFB961E753420E0A34FFF3FF4198CB5DB8A714AC10FC932
78AFDA74D86C6B7687F0880856903152A206D743D2E8C8B89CDE2EE9157A351
040E3042920EF86CC5CD978410587CF3932A323BB99F1C4BE6F7116003889EA
ACE54E43ADC1A417B94E230F494E1BE71042F398853248FC2C0578BDBA94C2
BBF09265F8120BCE70EE3B4ABDD9B08C23415013EBBD244991862E761BC6F9
36AC17B99377A3564ACBCD791B276F074875B94FDC4DF3F5C73F40C124DC0C
68E034260B71D445CE80CAC4EB67C5EB6A2DCA92538722C45AF652D5A676C8
A7867DDC8C7614A3D516C0D49359948B8915FB6588B5B221702ECAB4C65B083
DC399E7BBDA8F77F68165B9D4E922495C7B1F6FFA67ED0CE192F893ADE5F8C
08FC6D7E62199102D8674C97003A2DCACD29674F2782082D7B9FF4152BDAFB
FDBC9B52D42A05286C930FF4AAED9E037FAFE82D13439EA2DD1D8855038D73
73B0E517EB71BC0B34A3FB449EFAA7C21E8AB2F2420C8BDC398157B670B3EC
8167C3FFA07ADB5FA31034AD2C5E1DD93E8E6825BC3DA3276A92E59C6901AA
9197C0C85173778DEA

表1-18 蓝牙 SPP 起始报文示例

SOF (1Byte)	CT RL (1Byte)	LEN (2Byte)	FID (1Byte)	SN (1Byte)	Value (NByte)	CRC (2Byte)
0xAF	0x07	0xF801	0x24	0x00	0D0484027306170303066E268355074CCC17E21CA33FDDC85D7F02897E33F803758F2F07384DABA3B128526343868712BB58ECDC9369AC621510CC33756094CDDDB44FF81E28BB1F12AC974A5E7553CBCADFDDB6	0x8DEA

SOF (1Byte)	CT RL (1Byte)	LEN (2Byte)	FID (1Byte)	SN (1Byte)	Value (NByte)	CRC (2Byte)
					9504004FCB0F6CBB713D3BC4 3AFB961E753420E0A34FFF3F F4198CB5DB8A714AC10FC932 78AFDA74D86C6B7687F08808 56903152A206D743D2E8C8B8 9CDE2EE9157A351040E30429 20EF86CC5CD978410587CF39 32A323BB99F1C4BE6F711600 3889EAACE54E43ADC1A417B 94E230F494E1BE71042F39885 3248FC2C0578BDF0A94C2BBF0 9265F8120BCE70EE3B4ABDD 9B08C23415013EBBD2449918 62E761BC6F936AC17B99377A 3564ACBCD791B276F074875B 94FDC4DF3F5C73F40C124DC 0C68E034260B71D445CE80CA C4EB67C5EB6A2DCA9253872 2C45AF652D5A676C8A7867DD C8C7614A3D516C0D49359948 B8915FB6588B5B221702ECAB 4C65B083DC399E7BBDA8F77 F68165B9D4E922495C7B1F6F FA67ED0CE192F893ADE5F8C 08FC6D7E62199102D8674C97 003A2DCACD29674F2782082D 7B9FF4152BDAFBFDBC9B52D 42A05286C930FF4AAED9E037 FAFE82D13439EA2DD1D88550 38D7373B0E517EB71BC0B34A 3FB449EFAA7C21E8AB2F2420 C8BDC398157B670B3EC8167C 3FFA07ADB5FA31034AD2C5E 1DD93E8E6825BC3DA3276A92 E59C6901AA9197C0C8517377	

【蓝牙 SPP 过程报文帧 1 示例】

蓝牙 SPP 过程报文示例 1 (手机 APP→板侧的数据帧):

AF0BF801240152404654BF7A940E0706B7F6CF752CA11DFAB3A9FCD949A095F
0EC872D37157CE2D04C3A659C21306EAA4810925B2D0E9455BC160735876B3A
BB356B5B236D77F4896E947BD067CEFFFCF4DE7DCABA5152B8C0870F5284C5
90241B6B1062D44147CF1C77081A4C233F529DDEAA51C07700817145058928C7
7938BD3D4ECE07467BB961BFEF465967FE45ACDF11C14BCCD491ECE3421DB
8CBBE9EA4A4AE1329FC33FA227BB37A927BFDE6BEE93501C046D7F0489A791

73C3C995A35C85AC81D9A27357D8DD193A7556DC89E42BEF51A3DD6561A30D
 CDE2CFAB982A8F24B0E5677A7B64EE0DAA4A60F83E1FB8521902F8D8F4DC96
 3508011D73B2B7647A97E0056C8670539E3FA788D6594C07838D0F34F15A18AE
 ABF5EECA0C956BEC5811D25F1B82E7DA9DFA00B52FBBF0BEF829844144A7D
 B0B5FFDD31725121B76CA5171BAD6A41A158EB50B93A48F47DBED738C3DED
 C229612B57E7506DD7F0B8029E0D0C2B2DBBA2DB4FB0560231BC72542702A2
 7395F97FF3E8FAE54BFB2488A2863783FA0A4D0FEDE5F349673BADAE7FB2DE
 222F0D58634E173D5FC9DFFF7CEEC179DD941A74BDD09A9209C7AF38F85657
 78261ACEEDCCE1BFEE4BCCA03D2BFD04EFDB5D082C70E06875C673C62DAB
 179FB96F83AB7001A94DDB932DF6DA6D43755882278CBCDB5E09BF7FD550AA
 217579EFB1FED11B0FA9AEA

表1-19 蓝牙 SPP 过程报文示例 1

S O F (1 By te)	CT RL (1 By te)	LE N (2 By te)	FI D (1 By te)	S N (1 By te)	Value (NByte)	C R C (2 By te)
0x AF	0x 0B	0x F8 01	0x 24	0x 01	52404654BF7A940E0706B7F6CF752CA11DFAB 3A9FCD949A095F0EC872D37157CE2D04C3A65 9C21306EAA4810925B2D0E9455BC160735876B 3ABB356B5B236D77F4896E947BD067CEFFFCF 4DE7DCABA5152B8C0870F5284C590241B6B10 62D44147CF1C77081A4C233F529DDEAA51C07 700817145058928C77938BD3D4ECE07467BB96 1BFEF465967FE45ACDF11C14BCCD491ECE34 21DB8CBBE9EA4A4AE1329FC33FA227BB37A9 27BFDE6BEE93501C046D7F0489A79173C3C99 5A35C85AC81D9A27357D8DD193A7556DC89E 42BEF51A3DD6561A30DCDE2CFAB982A8F24B 0E5677A7B64EE0DAA4A60F83E1FB8521902F8 D8F4DC963508011D73B2B7647A97E0056C867 0539E3FA788D6594C07838D0F34F15A18AEAB F5EECA0C956BEC5811D25F1B82E7DA9DFA00 B52FBBF0BEF829844144A7DB0B5FFDD317251 21B76CA5171BAD6A41A158EB50B93A48F47DB ED738C3DEDC229612B57E7506DD7F0B8029E 0D0C2B2DBBA2DB4FB0560231BC72542702A27 395F97FF3E8FAE54BFB2488A2863783FA0A4D 0FEDE5F349673BADAE7FB2DE222F0D58634E 173D5FC9DFFF7CEEC179DD941A74BDD09A92 09C7AF38F8565778261ACEEDCCE1BFEE4BCC A03D2BFD04EFDB5D082C70E06875C673C62D AB179FB96F83AB7001A94DDB932DF6DA6D43 755882278CBCDB5E09BF7FD550AA217579EFB 1FED11B0FA	0x 9A EA

【蓝牙 SPP 过程报文帧 2 示例】

蓝牙 SPP 过程报文示例 2 (手机 APP→板侧的数据帧):

AF0BF80124025256E9442F9E7A542357EE31C3F7F295AB2540BFCF6C2D636CAB932D1333EE915702CF39C6449E861A848ACD340CCFEAB54C97FD63F350CB32852E771A3A2F5234895A6B95C7E8F1E4095288595A802255FE27B27B68312733DFACDDBBAF8D46C879449DF5D28EDC5E857878280CB25B2E890E151CE83B0A7EB21B60DA7EA870E7499B396C931F6EB413352CA5D65D2B5574A67A01A391C113928FF3E9F685D6BA1FC58D8860724D208BA03A68E8FA067D434FF740AD8564D9F9924B316D6456FBB892A2200A63B67B7D2CD87665559E1B5AA562CD6FA91D7FA3F75CBBAF84475CF059327DBFF55BEE674FA55ECE5C3920F9400F61DE7967632181D8C3DCDB519745D9847318466015B71D74ACA77FE0FA5A5A677623ECF6CFE4EDEF4F5F297F6A38180DA87E08200981091370E39C5477653C4635183FBE0E02DA3F158E85FD984B6A6BC6E6B7A9156BF62A9A12477A6BC9403412E6FF3EA310F163E3E46898D0F22BF4B26E32391FAE1B79B8912794ECB8C28510EBB7BFB06470860D9E20288150F50C0C8E20184CA26026BE1583802111266F1AD3484B33CD8445A395E13D850ADF71FB5F850512EF4872D8D4F0BD3BF768B968412F41609F4D89B3B6338C891AA3B698B235EC0E6FACAE7EDD595376D9BEDBDD13751B816C1E6D7E25A83B0519214053FBE3A82B33AE4C4EC1C34F4C263A9087C

表1-20 蓝牙 SPP 过程报文示例 2

S O F (1 By te)	C T R L (1 B y t e)	LE N (2 By te)	FI D (1 By te)	S N (1 By te)	Value (NByte)	C R C (2 By te)
0x A F	0 x 0 B	0x F8 01	0x 24	0x 02	5256E9442F9E7A542357EE31C3F7F295AB2540BFCF6C2D636CAB932D1333EE915702CF39C6449E861A848ACD340CCFEAB54C97FD63F350CB32852E771A3A2F5234895A6B95C7E8F1E4095288595A802255FE27B27B68312733DFACDDBBAF8D46C879449DF5D28EDC5E857878280CB25B2E890E151CE83B0A7EB21B60DA7EA870E7499B396C931F6EB413352CA5D65D2B5574A67A01A391C113928FF3E9F685D6BA1FC58D8860724D208BA03A68E8FA067D434FF740AD8564D9F9924B316D6456FBB892A2200A63B67B7D2CD87665559E1B5AA562CD6FA91D7FA3F75CBBAF84475CF059327DBFF55BEE674FA55ECE5C3920F9400F61DE7967632181D8C3DCDB519745D9847318466015B71D74ACA77FE0FA5A5A677623ECF6CFE4EDEF4F5F297F6A38180DA87E08200981091370E39C5477653C4635183FBE0E02DA3F158E85FD984B6A6BC6E6B7A9156BF62A9A12477A6BC9403412E6FF3EA310F163E3E46898D0F22BF4B26E32391FAE1B79B8912794ECB8C28510EBB7BFB06470860D9E20288150F50C0C8E20184CA26026BE1583802111266F1AD3484B3	0x 08 7 C

SOF (1 Byte)	CTRL (1 Byte)	LEN (2 Byte)	FID (1 Byte)	SN (1 Byte)	Value (NByte)	CRC (2 Byte)
					3CD8445A395E13D850ADF71FB5F850512EF4872 D8D4F0BD3BF768B968412F41609F4D89B3B6338 C891AA3B698B235EC0E6FACAE7EDD595376D9B EDBD13751B816C1E6D7E25A83B0519214053FB E3A82B33AE4C4EC1C34F4C263A9	

【蓝牙 SPP 结束报文帧示例】

蓝牙 SPP 结束报文示例 (手机 APP→板侧的数据帧):

AF0FA1002403D98ED69E73EA7E2906F56776E910964A4A463E640D803E0B9DC
E0CCB88C41079A662198EB905FD8745263C7B2A322996AEA7975D453D1A273E
5476AD829D411D025684C1B2C28B5022766C9091CDCE438A99ABF1D9D4C6E3
761B11EC3E11ABD396C1887689A0B070D5EB078D1A643959727647AA7F58A75
A449389AAC1DCFCB642629004E0DC645C048EFD65E7CE32FC1591CE67AB2F8
496411A8B2FE10BCB

表1-21 蓝牙 SPP 结束报文示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	SN 号	Value (NByte)	CRC (2Byte)
0xAF	0x0F	0xA100	0x24	0X03	D98ED69E73EA7E2906 F56776E910964A4A463 E640D803E0B9DCE0C CB88C41079A662198E B905FD8745263C7B2A 322996AEA7975D453D 1A273E5476AD829D41 1D025684C1B2C28B50 22766C9091CDCE438A 99ABF1D9D4C6E3761B 11EC3E11ABD396C188 7689A0B070D5EB078D 1A643959727647AA7F5 8A75A449389AAC1DCF CB642629004E0DC645 C048EFD65E7CE32FC 1591CE67AB2F849641	0x0BCB

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	SN 号	Value (NByte)	CRC (2Byte)
					1A8B2FE1	

分帧示例代码逻辑如下：

```
/**
 * 阿里socket服务端返回数据需要分帧处理
 *
 * @param responseData 收到阿里服务器返回的数据
 * @param splitLength 分帧字节长度
 * @return 字节数组列表
 */
public static List<byte[]> createTcpResponseFrame(byte[] responseData, int splitLength) {
    byte[] payload = getTcpResponseFrame(responseData);
    //首先根据阿里服务器返回的数据组成完成的数据帧
    byte[] payloadTlv = getMCAndTLV(
        CommonConstants.BT_RIDE_CODE_MID,
        CommonConstants.BT_RIDE_CODE_CID,
        CommonConstants.BT_SOCKET_TCP_RESPONSE_TYPE,
        payload
    );
    byte[] newPayload = new byte[payloadTlv.length - 1];
    System.arraycopy(payloadTlv, 1, newPayload, 0, newPayload.length);
    /*调用getNeedSendCmdList接口执行分帧操作，该接口的返回值为上述示例中的起始帧、过程
    帧、结束帧*/
    return getNeedSendCmdList(newPayload, splitLength);
}

附：分帧所调用到相关的接口如下所示：

/**
 * 帧格式指令的状态
 *
 * @since 2023-12-05
 */
public enum FrameStatus {
    NO_FRAME,
```

```

    FRAME_START,
    FRAME_EXECUTE,
    FRAME_END
}

/**
 * 外部调用的获取完整携带sn的命令列表
 *
 * @param payloadValue 携带id,mid,cid,以及多个tlv的字节数组
 * @param splitLength 分割长度
 * @return List<byte [ ]>
 */
public static List<byte[]> getNeedSendCmdList(byte[] payloadValue, int splitLength) {
    List<byte[]> completeCmdByteList = new ArrayList<>();
    List<byte[]> payloadValueList = fixedLengthSplitByteArray(payloadValue, splitLength);
    FrameStatus frameStatus;
    for (int i = 0; i < payloadValueList.size(); i++) {
        byte[] containsSrcAndSnAndTlvByteList = getContainsSNByteList(payloadValueList.get(i),
i);
        byte[] containsSrcAndSnAndTlvByteListCrc =
Utils.crc16(containsSrcAndSnAndTlvByteList);
        if (i == 0) {
            frameStatus = FrameStatus.FRAME_START;
        } else if (i == payloadValueList.size() - 1) {
            frameStatus = FrameStatus.FRAME_END;
        } else {
            frameStatus = FrameStatus.FRAME_EXECUTE;
        }
        byte[] tempFrame = getContainsScCompleteFrame(containsSrcAndSnAndTlvByteListCrc,
containsSrcAndSnAndTlvByteList, frameStatus);
        completeCmdByteList.add(tempFrame);
    }
    return completeCmdByteList;
}

/**
 * 按照定长切割字节数组
 *
 * @param byteArray TLV中的V
 * @param splitLength 定长

```

```

* @return List<byte [ ]>
*/
public static List<byte[]> fixedLengthSplitByteArray(byte[] byteArray, int splitLength) {
    int count = (int) Math.ceil((double) byteArray.length / splitLength);
    List<byte[]> hasSplitByteList = new ArrayList<>();
    int start = 0;
    for (int i = 0; i < count; i++) {
        int end = Math.min(start + splitLength, byteArray.length);
        hasSplitByteList.add(Arrays.copyOfRange(byteArray, start, end));
        start += splitLength;
    }
    return hasSplitByteList;
}

/**
 * 获取携带sn字段的分割指令
 *
 * @param hasSplitItemByteArray 已经被切割好的定长字节数组
 * @param index 被切割的定长字节数组的索引
 * @return byte[]
 */
public static byte[] getContainsSNByteList(byte[] hasSplitItemByteArray, int index) {
    byte[] resultByteList = new byte[1 + 1 + hasSplitItemByteArray.length];
    resultByteList[0] = (byte) 0x24;
    resultByteList[1] = getSnFrame(index);
    System.arraycopy(hasSplitItemByteArray, 0, resultByteList, 2, hasSplitItemByteArray.length);
    return resultByteList;
}

/**
 * 计算完整命令的crc16算法
 *
 * @param data byte[]
 * @return byte[] 两个字节
 */
public static byte[] crc16(byte[] data) {
    int crc = 0x0000;
    for (byte b : data) {
        crc = (crc << 8) ^ CYCLIC_REDUNDANT_CHECK_16_TAB[((crc >> 8) ^ (int) b) & 0xff];
    }
    return new byte[] {(byte) (0xff & crc), (byte) ((0xff00 & crc) >> 8)};
}

```

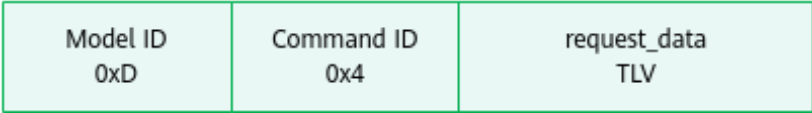
```
}
/**
 * 生成携带sn的完成帧格式命令
 *
 * @param crc          crc16校验位
 * @param lenValue     从fid开始至crc之前的所有内容
 * @param frameStatus  帧状态
 * @return byte[]
 */
private static byte[] getContainsScCompleteFrame(byte[] crc, byte[] lenValue, FrameStatus
frameStatus) {
    byte[] buf = new byte[2 + 2 + lenValue.length + 2];
    byte[] len = Utils.shortToLittleByteList((short) (lenValue.length + crc.length));
    buf[0] = getSofFrame();
    buf[1] = getCtrlFrame(frameStatus, true, true);
    System.arraycopy(len, 0, buf, 2, 2);
    System.arraycopy(lenValue, 0, buf, 4, lenValue.length);
    System.arraycopy(crc, 0, buf, buf.length - 2, crc.length);
    return buf;
}
```

1.3.4.4.6 TCP 上网请求的数据指令（单板->手机）

表1-22 TCP 上网请求字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
request_ data	0x04	var	uint8	此处为 TCP 的上 网请求数据。	M	1.0/1.0

- 请求帧结构：



- 应答帧结构：

- 返回成功：

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 100000
-----------------	-------------------	--------------	----------------	-----------------

- 返回失败:

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 1XXXXX
-----------------	-------------------	--------------	----------------	-----------------

解析出 value 数据, 调用 socket 发送指令接口发送给阿里服务器。

```
receivedTcpRequestHandler(commandParse.getPayload());
private void receivedTcpRequestHandler(byte[] payload) {
    //payload为表2中的value字段
    socketClient.senddataToserver(payload);
}
```

蓝牙 SPP 发送 TCP 请求报文示例 (板侧→手机 APP 的数据帧):

AF 03 14 01 40 0D 04 84 02 0B 01 16 03 03 01 06 10 00 01 02 01 00 49 4F A2 C0
 FB E3 89 0B D6 05 2C B7 67 84 F8 88 A5 53 F6 20 A5 90 CE 14 1C 7A 74 D8 60
 08 51 71 46 BC 95 85 CA 8C CC 6F 7D 96 DB B5 CA FE 65 DE 77 5B 7B 7E E4 24
 D2 A3 FE B2 F8 20 E5 A3 E1 BC D1 E4 4B 55 CA 19 6B DB 37 C4 36 CD BB CA
 60 46 1B 57 44 DC 34 82 E5 79 1A 38 4F 30 B1 85 C6 D4 74 08 1F 52 0F 34 32 CF
 DA 23 13 BC 58 EE 2C 2C E9 C8 27 4F 2F 33 D3 39 65 B8 75 A8 81 25 D6 0D E5
 1B 4D 7A BD 9B 99 EB 4C 22 AB 32 A5 AF 20 66 00 4B B4 3B BA 6A 16 C8 0C
 A0 4D 8D 6A BE ED AE D0 CB 28 7B 14 2A 00 6B 4A 57 EB 44 8A 4F B3 51 33 4D
 3A FC E9 C1 55 9B 90 B4 96 29 12 67 49 CB BE 82 E7 70 B2 72 28 AF 63 08 C9
 AA A3 7A 52 02 40 DC FF 06 A2 3E 78 39 76 52 D8 A4 61 EC 42 54 4B 57 E7 8D
 14 14 6F 5C 2B 3F EE 20 FC 67 48 4E 08 D4 8D 8D 19 49 95 2A 31 66 3A 47 24 9F
 FF 75 D7 F1

表1-23 蓝牙 SPP 发送 TCP 请求报文示例

S O F (1 B y t e)	C T R L (1 B y t e)	L E N (2 B y t e)	F I D (1 B y t e)	M o d e l I D	C o m m a n d I D	T y p e	l e n	value	CRC
0x A F	0x 03	0x 14 01	0x 40	0x 0 D	0x 04	0x 84	0x 02 0 B 01	0x16 03 03 01 06 10 00 01 02 01 00 49 4F A2 C0 FB E3 89 0B D6 05 2C B7 67 84 F8 88 A5 53 F6 20 A5 90 CE 14 1C 7A 74 D8 60 08 51 71 46 BC 95 85 CA 8C CC 6F 7D 96 DB B5 CA FE 65 DE 77 5B 7B 7E E4 24 D2 A3 FE B2 F8 20 E5 A3 E1 BC D1 E4 4B 55 CA 19 6B DB 37	0xD 7 F1

S O F (1 B y t e)	C T R L (1 B y t e)	L E N (2 B y t e)	F I D (1 B y t e)	M o d e l I D	C o m m a n d I D	T y p e	l e n	value	CRC
								C4 36 CD BB CA 60 46 1B 57 44 DC 34 82 E5 79 1A 38 4F 30 B1 85 C6 D4 74 08 1F 52 0F 34 32 CF DA 23 13 BC 58 EE 2C 2C E9 C8 27 4F 2F 33 D3 39 65 B8 75 A8 81 25 D6 0D E5 1B 4D 7A BD 9B 99 EB 4C 22 AB 32 A5 AF 20 66 00 4B B4 3B BA 6A 16 C8 0C A0 4D 8D 6A BE ED AE D0 CB 28 7B 14 2A 00 6B 4A 57 EB 44 8A 4F B3 51 33 4D 3A FC E9 C1 55 9B 90 B4 96 29 12 67 49 CB BE 82 E7 70 B2 72 28 AF 63 08 C9 AA A3 7A 52 02 40 DC FF 06 A2 3E 78 39 76 52 D8 A4 61 EC 42 54 4B 57 E7 8D 14 14 6F 5C 2B 3F EE 20 FC 67 48 4E 08 D4 8D 8D 19 49 95 2A 31 66 3A 47 24 9F FF 75	

1.3.4.4.7 关闭 socket 蓝牙指令处理流程（单板->手机）

表1-24 服务器 TCP 响应数据字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
request_ type	0x05	1	uint8	请求类型。 - value 值为 1: 手机发起 关闭。 - value 值为 2: 手表发起 关闭。	M	1.0/1.0

- 请求帧结构:

Model ID 0xD	Command ID 0x4	request_type TL
-----------------	-------------------	--------------------

- 应答帧结构:

- 返回成功:

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 100000
-----------------	-------------------	--------------	----------------	-----------------

- 返回失败:

Model ID 0xD	Command ID 0x4	Type 0xFF	Length 0x04	Value 1XXXXX
-----------------	-------------------	--------------	----------------	-----------------

蓝牙 SPP 发送关闭 socket 请求帧示例 (板侧→手机 APP 的数据帧):

AF 03 0B 00 40 0D 04 85 84 01 00 00 00 38 82

表1-25 蓝牙 SPP 发送关闭 socket 请求帧示例

SOF (1Byte)	CTRL (1Byte)	LEN (2Byte)	FID (1Byte)	Model ID (1Byte)	Command ID (1Byte)	Type (1Byte)	len (1Byte)	Value (NByte)	CRC (2Byte)
0xAF	0x03	0x0B00	0x40	0x0D	0x04	0x85	0x84	0x 01 000000	0x38 82

步骤 1 设置停止接收 socket 服务器参数、关闭 socket 接口。

```
private void receivedCloseHandler(byte[] payload) {
    socketClient.setLoopReveiveData(false);
    socketClient.closeClientSocket();
}
```

步骤 2 socket 关闭成功失败的状态组装成蓝牙指令返回给手表端。

----结束

1.3.4.4.8 收到指令成功应答帧处理逻辑

应答帧结构:

- 返回成功:

Model ID 0xE	Command ID 0x1	Type 0xFF	Length 0x04	Value 100000
-----------------	-------------------	--------------	----------------	-----------------

- 返回失败:

Model ID 0xE	Command ID 0x1	Type 0xFF	Length 0x04	Value 1XXXXX
-----------------	-------------------	--------------	----------------	-----------------

收到指令成功应答帧处理逻辑的示例代码如下所示:

```
private void commandSendingResultHandler(byte[] payload) {
    CommandParse.ParseCommonResponseFrame commonResponseFrame = new
CommandParse
    .ParseCommonResponseFrame(payload);
    String infoMsg;
    int colorSelect;
    if (commonResponseFrame.getResult()) {
        infoMsg = "命令执行成功";
        colorSelect = GREEN;
    } else {
        infoMsg = "命令执行失败";
        colorSelect = RED;
    }
    Utils.sendMessage(EXECUTE_LOG, infoMsg, handler, colorSelect, 0);
}
```

上面用到的函数接口如下: 解析出 value 字段, 10000 为执行成功, 否则执行失败。

```
/**
 * 手表返回的网络状态返回
 * network、tcp close均可使用
 */
public static class ParseCommonResponseFrame {
    byte[] payload;

    public ParseCommonResponseFrame(byte[] payload) {
        this.payload = payload;
    }
}

/**
 * 获取执行结果
 *
 * @return boolean
 */
```

```
public boolean getResult() {  
    int result = Utils.byteListToLittleInt(payload);  
    return result == 100000;  
}
```

1.4 手机端数传适配指导

文件数传业务指导说明请参见《穿戴类产品对外交互协议》的“5.6 文件传输”章节。

1.4.1 蓝牙数传入口 Activity 页面

蓝牙 SPP 和蓝牙 BLE 调用总入口共用同一个 Activity：
WearTest\app\src\main\java\com\wrtm\weartest\componentsactivity\FileTransferActivity.java

1.4.1.1 蓝牙 SPP 数传 Activity 调用示例

须知

调用 SPP 数传的前提是蓝牙 BR 已连接成功。

SPP 数传场景，调用 FileTransferActivity 示例代码和传参如下：

```
Intent intent = new Intent(mContext, FileTransferActivity.class);  
intent.putExtra("deviceType", 1);  
startActivity(intent);
```

参数说明如表 1-26 所示。

表1-26 SPP 数传参数说明

SPP 数传传参名称	参数说明
deviceType	如果是 BR 数传，参数为 1。 如果是 BLE 数传，参数为 2。

1.4.1.2 蓝牙 BLE 数传 Activity 调用示例

须知

调用 BLE 数传的前提是蓝牙 BLE 已连接成功。

BLE 数传场景，调用 FileTransferActivity 示例代码和传参如下：

```
Intent intent = new Intent(BleConnectedActivity.this, FileTransferActivity.class);
intent.putExtra("deviceType", 2);
intent.putExtra("sID", transSid);
intent.putExtra("cID", transCid);
startActivity(intent);
```

参数说明如表 1-27 所示。

表1-27 BLE 数传参数说明

BLE 数传传参名称	参数说明
deviceType	如果是 BR 数传，参数为 1。 如果是 BLE 数传，参数为 2。
sID	BLE 服务的 ServicesUUID。
cID	BLE 服务的 CharacteristicsUUID。

1.4.1.3 数传上下行页面适配器示例

数传上下行页面适配器代码路径：

\\WearTest\\app\\src\\main\\java\\com\\wrtm\\weartest\\adapter\\FileTransferPagerAdapter.java

上传下载页面类如表 1-28 所示。

表1-28 上传下载页面说明

页面类别	类名称	代码路径
数传上传页面	FileUploadFragment	WearTest\\app\\src\\main\\java\\com\\wrtm\\weartest\\componentsfragment
数传下载页面	FileDownloadFragment	WearTest\\app\\src\\main\\java\\com\\wrtm\\weartest\\componentsfragment

在适配器中会给页面传参数类型，在 BLE 场景下会传参 ServicesUUID 和 CharacteristicsUUID 参数，接口说明如表 1-29 所示。

表1-29 上传下载页面传参说明

接口名称	接口描述	参数说明
setDeviceType	设置设备类型。	如果是 BR 数传，参数为 1。 如果是 BLE 数传，参数为 2。
setBleTransferUUID	设置 BLE 设备的 ServicesUUID 和 CharacteristicsUUID。	sId: ServicesUUID。 cId: CharacteristicsUUID。

1.4.2 蓝牙上传场景代码调用说明

SPP 数传和 BLE 数传上传共用 FileUploadFragment.java 页面，主要调用数传上传线程，调用上传示例代码如下面章节所述。

1.4.2.1 定义蓝牙文件上传回调函数

定义蓝牙数传上传回调函数（回调函数接口里为示例代码）：

```
private final DataTransferCallback dataTransferCallback = new DataTransferCallback() {  
    @Override  
    public void onStart() {  
        Utils.sendMessage(SEND_FILE_INFO, null, fileHandler);  
        if (deviceType == FileTransferActivity.BLE_DEVICE) {  
            bleManager.setSendingStatus(true);  
            bleManager.sendFileTransferUploadThread(UUID.fromString(sId),  
            UUID.fromString(cId));  
            bleManager.setBleConnectListener(onBleConnectListener);  
        }  
    }  
    @Override  
    public void onFinish(String... args) {  
        if (deviceType == FileTransferActivity.BLE_DEVICE) {  
            bleManager.setSendingStatus(false);  
        }  
    }  
}
```

```
        Utils.sendMessage(SEND_OVER, null, fileHandler);
    }
    @Override
    public void onStop() {
        Utils.sendMessage(SEND_STOPED, null, fileHandler);
    }
    @Override
    public void onError(String errorMsg) {
    }
    @Override
    public void onExecute(double progress, long rate, long sentSize, long totalSize) {
        Utils.sendMessage(SEND_REPLY_INFO, (int) progress, fileHandler, (int) rate, 0);
    }
    @Override
    public void sendCmd(byte[] cmd) {
        sendToBoardCmd(cmd);
    }
}
};
```

蓝牙文件上传回调函数接口说明如表 1-30 所示。

表1-30 蓝牙文件上传回调函数接口说明

回调接口名称	接口描述	参数说明	WRTM APK 示例代码中添加的功能说明	是否在回调接口里必须添加操作
onStart	标识数传开始会进入此函数：用户可以在此添加数传开始需要执行的操作。	无	在 BLE 数传场景下，设置循环发送数据为 true，调用发送数据线程功能，注册 BLE 回调。	可选，用户根据实际代码情况判断是否需要添加。
onFinish	标识数传执行结束：用户可以在此添加数传结束需要执行的操作。	无	设置循环发送数据状态为 false，调用 handler 状态用于在页面上显示上传结束。	可选，用户根据实际代码情况判断是否需要添加。

回调接口名称	接口描述	参数说明	WRTM APK 示例代码中添加的功能说明	是否在回调接口里必须添加操作
onStop	上传异常终止标识：用户可以在此添加数传异常终止后的操作。	无	调用 handler 状态用于在页面上显示上传异常终止。	可选，用户根据实际代码情况判断是否需要添加。
onError	未收到协商帧或参数校验错误无法进行数传标识：用户可以在此添加数传异常终止后的操作。	errorMsg：失败原因。	失败原因可能为： - 重试 3 次未收到协商帧 - 参数校验错误	可选，用户根据实际代码情况判断是否需要添加。
onExecute	用户传输进度和速率标识：用户可以在此添加需要执行的操作。	- Progress：传输进度。 - Rate：传输速率。 - sentSize：已传输字节大小。 - totalSize：总共需要传输的字节大小。	调用 handler 状态用于在页面上实时显示传输进度和速率消息。	可选，用户根据实际代码情况判断是否需要添加。
sendCmd	需要实现蓝牙发送指令的功能。	cmd：需要发送的蓝牙指令。	调用 sendToBoardCmd(cmd)函数实现 BR/BLE 发送消息功能	必选，需要实现 BR/BLE 发送指令接口。

1.4.2.2 实例化蓝牙文件上传类

实例化蓝牙文件上传类 DataUploadTransfer，调用 start 执行上传任务。

- BLE 调用示例代码如下：

```
dataUploadTransfer = new
DataUploadTransfer().setDeviceType(DataUploadTransfer.BLE_DEVICE)
    .setUploadFile(uploadFilePath)
    .setUploadPath(uploadFileToBoardPath)
    .setTransmitId(transmitId)
    .setUserDefineBlockSize(tempBlockSize)
    .setUserDefineBlockNum(tempBlockNum)
    .setDataTransferCallback(dataTransferCallback);
dataUploadTransfer.start();
```

- BR 文件上传调用示例代码如下：

```
dataUploadTransfer = new
DataUploadTransfer().setDeviceType(DataUploadTransfer.BR_DEVICE)
    .setUploadFile(uploadFilePath)
    .setUploadPath(uploadFileToBoardPath)
    .setTransmitId(transmitId)
    .setDataTransferCallback(dataTransferCallback);
```

蓝牙文件上传类参数说明如表 1-31 所示。

表1-31 蓝牙文件上传类参数说明

接口名称	接口描述	参数说明	是否必传
setDeviceType	设置数传上传任务是 BLE 类型还是 BR 类型。	如果是 BR 数传，参数为 1。 如果是 BLE 数传，参数为 2。	是
setUploadFile	设置数传上传文件手机侧的路径。	参数示例（手机上的路径）： /storage/emulated/0/androidTestLogDir/WRTM_LOG/2024-01-12-14WRTM_Log.log	是
setUploadPath	上传到板侧的路径。	参数为《穿戴类产品对外交互协议》的“5.6.2 传输参数协商”章节协商帧的 info[0]字段。 例如：/system/	是

接口名称	接口描述	参数说明	是否必传
setTransmitId	标识本次数传任务的数传 ID。	参数为《穿戴类产品对外交互协议》的“5.6.2 传输参数协商”章节协商帧的 transmit_id 字段。 参数为 1、2 等。	是
setUserDefineBlockSize	每次传输大小（单位：Byte）。	参数为《穿戴类产品对外交互协议》的“5.6.2 传输参数协商”章节协商帧的 data_block_size 字段。 APK 文件上传设置为 470，用户可根据实际情况设置。	是
setUserDefineBlockNum	每组传输次数。	参数为《穿戴类产品对外交互协议》的“5.6.2 传输参数协商”章节协商帧的 data_block_number 字段。 APK 文件上传设置为 35，用户可根据实际情况设置。	是
setDataTransferCallback	上传状态回调函数。	传入“ 1.4.2.1 定义蓝牙文件上传回调函数 ”章节定义的回调函数。	是

1.4.2.3 实现蓝牙接收数据接口

- WRTM APK 蓝牙 BR 接收数据示例调用代码如下，用户需要在自己的 BR/BLE 接收蓝牙数据接口里调用（需要添加红色示例代码）：

```
private final BTDeviceBRService.OnSendReceiveDataListener
onUploadSendReceiveDataListener = new BTDeviceBRService
    .OnSendReceiveDataListener() {
    @Override
    public void onSendDataSuccess(byte[] data) {
    }
    @Override
    public void onSendDataError(byte[] data, String errorMsg) {
    }
}
```

```
@Override
public void onReceiveDataSuccess(byte[] buffer) {
    if (dataUploadTransfer != null && dataUploadTransfer.isRunning()) {
        dataUploadTransfer.addReceiveCacheByteList(buffer);
    }
}

@Override
public void onReceiveDataError(String errorMsg) {
}

};
```

- WRTM APK 蓝牙 BLE 接收数据示例调用代码如下（需要添加红色示例代码）：

```
@Override
public void onReceiveMessage(BluetoothGatt bluetoothGatt, BluetoothDevice
bluetoothDevice, BluetoothGattCharacteristic characteristic, byte[] msg) {
    if (dataUploadTransfer != null && dataUploadTransfer.isRunning()) {
        dataUploadTransfer.addReceiveCacheByteList(msg);
    }
}
```

1.4.2.4 停止蓝牙文件上传接口

需要执行停止数传上传操作时，调用此接口：

```
if (dataUploadTransfer != null) {
    dataUploadTransfer.stopRunning();
}
```

1.4.3 蓝牙下载场景代码调用说明

SPP 数传和 BLE 数传共用 FileDownloadFragment.java 页面，主要调用数传下载线程，调用下载示例代码如下。

1.4.3.1 定义蓝牙文件下载回调函数

请参见“[1.4.2.1 定义蓝牙文件上传回调函数](#)”章节。

1.4.3.2 实例化蓝牙文件下载类

实例化蓝牙文件下载类 DataDownloadTransfer，调用 start 执行下载任务。

- BLE 调用示例代码：

```
dataDownloadTransfer = new DataDownloadTransfer()
    .setUserDefineBlockSize(tempBlockSize)
    .setUserDefineBlockNum(tempBlockNum)
```

```
.setDeviceType(DataDownloadTransfer.BLE_DEVICE);  
.setTransmitId(transmitId)  
.setDataTransferCallback(dataTransferCallback)  
.setSaveLocalPath("androidTestLogDir/download")  
.setDownloadFile(downloadFile)  
.setDownloadFileTotalSize(downloadFileSize)  
.setFromBoardParentPath(downloadFromBoardPath);
```

● SPP 调用示例代码：

```
dataDownloadTransfer = new DataDownloadTransfer()  
.setDeviceType(DataDownloadTransfer.BR_DEVICE);  
.setTransmitId(transmitId)  
.setDataTransferCallback(dataTransferCallback)  
.setSaveLocalPath("androidTestLogDir/download")  
.setDownloadFile(downloadFile)  
.setDownloadFileTotalSize(downloadFileSize)  
.setFromBoardParentPath(downloadFromBoardPath);  
dataDownloadTransfer.start();
```

蓝牙文件下载类参数说明如表 1-32 所示。

表1-32 蓝牙文件下载类参数说明

接口名称	接口描述	参数说明	是否必传
setDeviceType	设置数传下载任务是 BLE 类型还是 BR 类型。	如果是 BR 数传，参数为 1。 如果是 BLE 数传，参数为 2。	是
setTransmitId	标识本次数传任务的数传 ID。	参数为《穿戴类产品对外交互协议》的“5.6.2 传输参数协商”章节协商帧的 transmit_id 字段。 参数为 1、2 等。	是
setDataTransferCallback	下载状态回调函数。	参考“1.4.2.1 定义蓝牙文件上传回调函数”章节定义的回调函数。	是
setSaveLocalPath	设置保存在手机侧的路径。	本章节 APK 参考参数示例为 androidTestLogDir/download。	是

接口名称	接口描述	参数说明	是否必传
setDownloadFile	板侧的文件名称。	例如板侧存在“/user/cai.map3”，则下载的话该参数传入 cai.map3。	是
setDownloadFileTotalSize	下载文件的大小（单位：Byte）。	例如需要下载的板侧文件大小为 608316Byte，那么该参数传入为 608316。	是
setFromBoardParentPath	下载板侧的路径。	比如从板侧 system 下载文件，则该参数传参“/system/”。	是
setUserDefineBlockNum	每组传输次数。	参数为《穿戴类产品对外交互协议》“5.6.2 传输参数协商”章节协商帧的 data_block_number 字段。 APK 文件下载设置为 20，用户可根据实际情况设置。	是
setUserDefineBlockSize	每次传输大小（单位：Byte）。	参数为《穿戴类产品对外交互协议》“5.6.2 传输参数协商”章节协商帧的 data_block_size 字段。 APK 文件下载设置为 470，用户可根据实际情况设置。	是

1.4.3.3 实现蓝牙接收数据接口

请参见“1.4.2.3 实现蓝牙接收数据接口”章节，在蓝牙接收数据接口里添加此函数：
dataDownloadTransfer.addReceiveCacheByteList(buffer);

- 用户需要添加 BR 蓝牙接收指令参考代码示例（添加代码为红色标识所示）：

```
private final BTDeviceBRService.OnSendReceiveDataListener btTransferCallback = new
BTDeviceBRService
    .OnSendReceiveDataListener() {
    @Override    public void onSendDataSuccess(byte[] data) {
    }
    @Override
```

```

public void onSendDataError(byte[] data, String errorMsg) {
}
@Override
public void onReceiveDataSuccess(byte[] buffer) {
    if (taskType == 1) {
        dataLsTransfer.addReceiveCacheByteList(buffer);
    } else if (taskType == 2) {
        dataDownloadTransfer.addReceiveCacheByteList(buffer);
    } else {
        dataDeleteTransfer.addReceiveCacheByteList(buffer);
    }
}
@Override
public void onReceiveDataError(String errorMsg) {
}
};

```

- 用户需要添加 BLE 蓝牙接收指令参考代码示例（添加代码为红色标识所示）：

```

@Override
public void onReceiveMessage(BluetoothGatt bluetoothGatt1, BluetoothDevice
bluetoothDevice,
BluetoothGattCharacteristic characteristic,
byte[] msg) {
    if (taskType == 1) {
        dataLsTransfer.addReceiveCacheByteList(buffer);
    } else if (taskType == 2) {
        dataDownloadTransfer.addReceiveCacheByteList(buffer);
    } else {
        dataDeleteTransfer.addReceiveCacheByteList(buffer);
    }
}
}

```

1.4.3.4 停止蓝牙文件下载接口

要执行停止数传下载操作时，调用此接口：

```

if (dataDownloadTransfer != null) {
    dataDownloadTransfer.stopRunning();
}

```

1.4.4 蓝牙查询文件列表接口说明

1.4.4.1 定义查询文件列表蓝牙回调函数

查询文件列表回调函数参考代码如下所示，代码参考路径：

WearTest\app\src\main\java\com\wrtm\weartest\componentsfragment\FileDownloadFragment.java

参考代码如下：

```
private final DataLsCallback dataLsCallback = new DataLsCallback() {  
    @Override  
    public void onStart() {  
        Utils.sendMessage(LS_START, null, downloadHandler);  
        if (deviceType == FileTransferActivity.BLE_DEVICE) {  
            bleManager.setSendingStatus(true);  
            bleManager.sendFileTransferUploadThread(UUID.fromString(sld),  
            UUID.fromString(cld));  
            bleManager.setBleConnectListener(onBleConnectListener);  
        }  
    }  
    @Override  
    public void onFinish(List<FileItemStruct> fileItemStructList) {  
        fileItemStructs.addAll(fileItemStructList);  
        Utils.sendMessage(LS_END, null, downloadHandler);  
        if (downloadLoadingDialog != null && downloadLoadingDialog.isShowing()) {  
            downloadLoadingDialog.dismiss();  
        }  
    }  
    @Override  
    public void onError(String errorMsg) {  
        Utils.sendMessage(ERROR, errorMsg, downloadHandler);  
    }  
    @Override  
    public void sendCmd(byte[] cmd) {  
        sendToBoardCmd(cmd);  
    }  
};
```

查询文件列表蓝牙回调函数接口说明如表 1-33 所示。

表1-33 查询文件列表蓝牙回调函数接口说明

回调接口名称	接口描述	参数说明	WRTM 示例代码中添加说明	是否在回调接口里必须添加操作
onStart	标识开始查询文件列表。	无	在 BLE 数传场景下，设置循环发送数据参数为 true，调用发送数据线程功能，注册 BLE 状态回调。	可选，用户根据实际代码情况判断是否需要添加。
onFinish	标识文件列表查询结束：用户可以在此添加文件查询结束需要执行的操作。	无	示例代码中包含以下功能： - 给文件列表显示适配器赋值参数（用于在页面上显示查询到的文件列表） <code>List<FileItemStruct> fileItemStructList</code>。 其中文件结构体参数如下： <code>isFile: True</code> <code>FileSize: 608316</code> <code>fileName: cai.mp3</code> - 取消查询等待提示框。	可选，用户根据实际代码情况判断是否需要添加。
onError	无法查询到文件列表的失败原因：用户可以在此添加数传异常终止后的操作。	errorMsg：失败原因。	示例代码中失败原因类型： - 参数校验失败。 20 秒超时仍未查询到文件信息。	可选，用户根据实际代码情况判断是否需要添加。
sendCmd	需要实现蓝牙发送指令的功能。	cmd：需要发送的蓝牙指令。	调用 <code>sendToBoardCmd(cmd)</code> 函数实现 BR/BLE 发送消息功能。	必选，需要实现 BR/BLE 发送指令接口。

1.4.4.2 实例化查询文件列表类

实例化查询文件列表示例代码：

```
dataLsTransfer = new DataLsTransfer()
    .setQueryBoardPath(downloadFromBoardPath)
    .setDataLsCallback(dataLsCallback);
dataLsTransfer.start();
```

表1-34 查询文件列表类参数说明

接口名称	接口描述	参数说明	是否必传
setQueryBoardPath	需要查询的板侧路径	例如： “/system/” 。	是
setDataLsCallback	查询文件列表状态回调函数	请参考 “ 1.4.4.1 定义查询文件列表蓝牙回调函数 ” 章节定义的回调函数。	是

1.4.4.3 实现蓝牙接收数据接口

参考 “[1.4.3.3 实现蓝牙接收数据接口](#)” 章节，在蓝牙接收数据接口里添加此函数：
dataLsTransfer.addReceiveCacheByteList(buffer);

- BR 蓝牙接收指令参考代码示例（添加代码为红色标识所示）：

```
private final BTDeviceBRService.OnSendReceiveDataListener btTransferCallback = new
BTDeviceBRService
    .OnSendReceiveDataListener() {
    @Override
    public void onSendDataSuccess(byte[] data) {
    }
    @Override
    public void onSendDataError(byte[] data, String errorMsg) {
    }
    @Override
    public void onReceiveDataSuccess(byte[] buffer) {
        if (taskType == 1) {
            dataLsTransfer.addReceiveCacheByteList(buffer);
        } else if (taskType == 2) {
            dataDownloadTransfer.addReceiveCacheByteList(buffer);
        }
    }
}
```



```
    } else {  
        dataDeleteTransfer.addReceiveCacheByteList(buffer);  
    }  
}  
@Override  
public void onReceiveDataError(String errorMsg) {  
}  
};
```

- BLE 蓝牙接收指令参考代码示例（添加代码为红色标识所示）：

```
@Override  
public void onReceiveMessage(BluetoothGatt bluetoothGatt1, BluetoothDevice  
bluetoothDevice,  
                             BluetoothGattCharacteristic characteristic, byte[] msg) {  
    if (taskType == 1) {  
        dataLsTransfer.addReceiveCacheByteList(buffer);  
    } else if (taskType == 2) {  
        dataDownloadTransfer.addReceiveCacheByteList(buffer);  
    } else {  
        dataDeleteTransfer.addReceiveCacheByteList(buffer);  
    }  
}
```

1.4.5 文件传输帧详解

本章节主要描述数传过程中的详细帧格式字段说明。

1.4.5.1 传输协商帧

1.4.5.1.1 协商帧字段详解

表1-35 传输协商帧字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可 选 (M/O)	版本
transmit _negotia te	0x01	sizeof(tran smi t_ne gotia te_p kt_t) + sizeof(inf o)	struct	• uint32_t transmit_id: 单 次传输唯一标 识。 • uint16_t src_send:1: 传	M	1.0/1.0

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
				输方向（占 1bit）。 - uint16_t struct_ver:2: 结构版本，助听器项目为 0（占 2bit）。 - uint16_t re_trans:1: 是否使能断电续传，助听器项目值为 0，不支持（占 1bit）。 - uint16_t bitmap_enable: 1: 是否使能位域，默认为 0（占 1bit）。 - uint16_t pad:11: 预留 bit（占 11bit）。 - uint16_t transmit_type: 交互类型，详情参考代码枚举值 transmit_type_t。 - uint32_t total_size: 文件大小。单位：Byte。 - uint16_t data_block_num: 每组传输次		

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可 选 (M/O)	版本
				数。 - uint16_t data_block_size: 每次传输大小。单位: Byte。 - uint32_t info_size: info 内容大小, 如文件路径名长度。单位: Byte。 - uint8_t info[0]: info 数据内容。		
transmit_negotiate_ack	0x02	sizeof(transmit_negotiate_ack_pkt_t) + sizeof(info)	struct	- uint32_t transmit_id: 单次传输唯一标识。 - uint16_t data_block_number: 每组传输次数。 - uint16_t data_block_size: 每次传输大小。单位: Byte。 - uint32_t info_size: info 内容大小。 - uint8_t info[0]: info 数据内容。	O	1.0/1.0

1.4.5.1.2 协商帧结构及示例

Model ID 0x06	Command ID 0x02	transmit_negotiate TLV
------------------	--------------------	---------------------------

传输上传协商帧与传输下载协商帧类似，区别之处在于传输方向字段。下面提供传输下载协商帧示例。

传输下载协商帧示例：

AF 03 3B 00 24 06 02 81 B4 01 00 00 00 00 30 00 00 00 2C 01 00 0E 00 C5 03 00
00 1F 00 1C 00 00 00 2F 75 73 65 72 2F 6C 6F 67 2F 78 69 61 6F 64 75 2F 78 69
61 6F 64 75 2E 70 63 6D 00 7C 38

表1-36 传输下载协商帧结构示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE (52Byte)	CR C (2B yte)
AF	03	3B 00	24	06	02	81	B4	01 00 00 00 00 30 00 00 00 2C 01 00 0E 00 C5 03 00 00 1F 00 1C 00 00 00 2F 75 73 65 72 2F 6C 6F 67 2F 78 69 61 6F 64 75 2F 78 69 61 6F 64 75 2E 70 63 6D 00	7C 38

1.4.5.1.3 协商应答帧结构及示例

传输下载协商应答帧示例：

AF 03 13 00 42 06 02 82 8C 01 00 00 00 0E 00 C5 03 00 00 00 00 CE 40

表1-37 传输下载协商应答帧结构示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE (12Byte)	CR C (2B yte)
AF	03	13 00	42	06	02	82	8C	01 00 00 00 0E 00 C5 03 00 00 00 00	CE 40

1.4.5.2 传输请求帧

1.4.5.2.1 请求帧字段详解

表1-38 传输请求帧字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
transmit _request	0x01	sizeof(tran smit_re quest_pkt _t)	struct	- uint32_t transmit_id: 单次传输唯一标识。 - uint32_t cnt: 默认为 1, 后续支持扩展传入多组 offset 和 size。 - uint32_t offset: 文件具体偏移值。 - uint32_t size: 基于文件偏移 offset 的传输大小值。单位: Byte。	M	1.0/1.0

1.4.5.2.2 传输请求帧结构及示例

Model ID 0x06	Command ID 0x03	transmit_request TLV
------------------	--------------------	-------------------------

传输请求帧示例：

AF 03 17 00 24 06 03 81 90 01 00 00 00 01 00 00 00 00 00 00 00 C6 34 00 00 84 7A

表1-39 传输下载协商帧结构示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TY PE (1B yte)	LEN (1B yte)	VALUE (16Byte)	CR C (2B yte)
AF	03	17 00	24	06	03	81	90	01 00 00 00 01 00 00 00 00 00 00 00 C6 34 00 00	84 7A

1.4.5.3 传输回应帧

1.4.5.3.1 传输回应帧字段详解

表1-40 传输回应帧字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
transmit _reply	0x01	sizeof(tran smmit_re ply_pkt_t) + sizeof(dat a)	struct	- uint32_t transmit_id: 单次传输唯一 标识。 - uint32_t offset: 文件 具体偏移值。	transmit_r epley	0x01

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
				- uint32_t size: 本次传 输大小值。单 位: Byte。 - uint8_t data[0]: 数据 内容。		

1.4.5.3.2 传输回应帧结构及示例

Model ID 0x06	Command ID 0x04	transmit_reply TLV
------------------	--------------------	-----------------------

传输回应帧示例:

AF 03 DA 03 42 06 04 81 02 D1 03 01 00 00 00 00 00 00 00 C5 03 00 00 00 00 01
00 02 00 03 00 03 00 03 00 03 00 02 00 01 00 01 00 01 00 00 00 FE FF FD FF FD
FF FC FF FA FF F9 FF F9 FF F8 FF F6 FF F5 FF F5 FF F3 FF F1 FF F1 FF F1
FF EF FF EE FF EE FF EE FF ED FF EC FF EC FF ED FF EC FF EA FF EA FF
EC FF ED FF EC FF ED FF EF FF F1 FF F1 FF F2 FF F4 FF F5 FF F3 FF F4 FF
F5 FF F5 FF F5 FF F7 FF FA FF FC FF FB FF FB FF FD FF FD FF FB FF FC FF
FE FF FE FF FC FF FD FF 00 00 00 00 FF FF 00 00 03 00 02 00 00 00 00 02
00 02 00 01 00 03 00 04 00 04 00 04 00 06 00 09 00 09 00 07 00 0A 00 0F 00 0F
00 0E 00 0E 00 12 00 16 00 17 00 19 00 1C 00 1C 00 1A 00 1B 00 20 00 24 00 25
00 28 00 2C 00 2E 00 2D 00 2F 00 32 00 34 00 34 00 36 00 3A 00 39 00 38 00 3A
00 3C 00 3B 00 38 00 38 00 39 00 35 00 30 00 2D 00 2C 00 28 00 23 00 23 00 23
00 1E 00 16 00 12 00 10 00 08 00 FE FF FA FF FA FF F9 FF F3 FF ED FF E8 FF
E2 FF DC FF DA FF D9 FF D3 FF CB FF C5 FF C3 FF BE FF B8 FF B6 FF B7 FF
B5 FF B1 FF B2 FF B6 FF B4 FF AF FF AE FF AF FF AE FF A9 FF A9 FF AE FF
AE FF AB FF AC FF B0 FF B5 FF B8 FF BC FF C1 FF C0 FF BE FF C1 FF C6 FF
C9 FF C9 FF CD FF D6 FF DB FF DC FF DF FF E8 FF EE FF F2 FF F6 FF FA FF
FD FF FB FF FD FF 04 00 08 00 0A 00 0E 00 16 00 1A 00 1A 00 1A 00 1E 00 1E
00 1C 00 1C 00 1F 00 20 00 22 00 28 00 2E 00 2F 00 2C 00 2E 00 34 00 33 00 31
00 34 00 39 00 3A 00 37 00 3C 00 42 00 41 00 3D 00 3D 00 41 00 40 00 3C 00 40
00 44 00 47 00 45 00 44 00 45 00 41 00 3A 00 37 00 33 00 30 00 2B 00 29 00 2B
00 28 00 23 00 1F 00 1E 00 1C 00 17 00 18 00 1A 00 17 00 12 00 10 00 11 00 0D
00 06 00 05 00 07 00 02 00 FB FF FC FF 01 00 FE FF F6 FF F3 FF F3 FF EE FF
E5 FF E1 FF E1 FF DD FF D5 FF D1 FF CF FF CA FF C3 FF C2 FF C2 FF BD FF
B1 FF AF FF B2 FF AE FF AA FF AA FF B2 FF B1 FF A9 FF A7 FF A9 FF A8 FF
A2 FF 9F FF A4 FF A4 FF 9F FF 9F FF A2 FF A2 FF 9F FF A2 FF AA FF B0 FF
B1 FF B5 FF BE FF C4 FF C6 FF CA FF CE FF CE FF D0 FF DC FF EB FF F0
FF F4 FF 00 00 0E 00 15 00 14 00 17 00 25 00 2D 00 A8 00 CA FF 95 00 F9 FF
7A 00 24 00 6C 00 57 00 5C 00 76 00 5E 00 90 00 62 00 8F 00 68 00 8B 00 6E 00
88 00 80 00 89 00 80 00 7D 00 86 00 7B 00 7A 00 70 00 73 00 67 00 62 00 61 00

63 00 58 00 4D 00 4F 00 52 00 4B 00 3F 00 3D 00 3F 00 39 00 2D 00 2A 00 2D
 00 2C 00 20 00 1C 00 18 00 0E 00 01 00 F9 FF FB FF F8 FF F1 FF F1 FF F1 FF
 F0 FF EB FF E8 FF EB FF E0 FF D5 FF D6 FF D7 FF D5 FF D1 FF D7 FF E5 FF
 DD FF CC FF C2 FF C1 FF BB FF AE FF AB FF B1 FF AF FF A7 FF AB FF B7
 FF B6 FF AA FF A6 FF AB FF AC FF A3 FF A3 FF A8 FF 9D FF 94 FF 9B FF A7
 FF A8 FF 9E FF 9E FF A3 FF 99 FF 90 FF 91 FF 9C FF A0 FF A0 FF AB FF BA
 FF BF FF BF FF BE FF C6 FF CA FF C6 FF CE FF DD FF E9 FF F2 FF FA FF 0D
 00 13 00 0F 00 1C 00 27 00 20 00 17 00 20 00 33 00 2C 00 1A 00 20 00 30 00 35
 00 2E 00 3B 00 51 00 4A 00 31 00 2C 00 32 00 29 00 14 00 0A 00 10 00 0D 00 02
 00 FF FF 08 00 08 00 01 00 FB FF FE FF 08 00 08 00 12 00 22 00 21 00 23 00 39
 00 57 00 66 00 6F 00 95 00 BD 00 C7 00 C4 00 DA 00 FE 4B 3B

表1-41 传输回应帧示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (3B yte)	VALUE (977Byte)	CR C (2B yte)
AF	03	DA 03	42	06	04	81	02 D1 03	01 00 00 00 00 00 00 00 C5 03 00 00 00 00 01 00 02 00 03 00 03 00 03 00 03 00 02 00 01 00 01 00 01 00 00 00 FE FF FD FF FD FF FC FF FA FF F9 FF F9 FF F8 FF F6 FF F5 FF F5 FF F3 FF F1 FF F1 FF F1 FF EF FF EE FF EE FF EE FF ED FF EC FF EC FF ED FF EC FF EA FF EA FF EC FF ED FF EC FF ED FF EF FF F1 FF F1 FF F2 FF F4 FF F5 FF F3 FF F4 FF F5 FF F5 FF F5 FF F7 FF FA FF FC FF	4B 3B

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (3B yte)	VALUE (977Byte)	CR C (2B yte)
								FB FF FB FF FD FF FD FF FB FF FC FF FE FF FE FF FC FF FD FF 00 00 00 00 FF FF 00 00 03 00 02 00 00 00 00 00 02 00 02 00 01 00 03 00 04 00 04 00 04 00 06 00 09 00 09 00 07 00 0A 00 0F 00 0F 00 0E 00 0E 00 12 00 16 00 17 00 19 00 1C 00 1C 00 1A 00 1B 00 20 00 24 00 25 00 28 00 2C 00 2E 00 2D 00 2F 00 32 00 34 00 34 00 36 00 3A 00 39 00 38 00 3A 00 3C 00 3B 00 38 00 38 00 39 00 35 00 30 00 2D 00 2C 00 28 00 23 00 23 00 23 00 1E 00 16 00 12 00 10 00 08 00 FE FF FA FF FA FF F9 FF F3 FF ED FF E8 FF E2 FF DC FF DA FF D9 FF D3 FF CB FF C5 FF C3 FF BE FF B8 FF B6 FF	

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (3B yte)	VALUE (977Byte)	CR C (2B yte)
								B7 FF B5 FF B1 FF B2 FF B6 FF B4 FF AF FF AE FF AF FF AE FF A9 FF A9 FF AE FF AE FF AB FF AC FF B0 FF B5 FF B8 FF BC FF C1 FF C0 FF BE FF C1 FF C6 FF C9 FF C9 FF CD FF D6 FF DB FF DC FF DF FF E8 FF EE FF F2 FF F6 FF FA FF FD FF FB FF FD FF 04 00 08 00 0A 00 0E 00 16 00 1A 00 1A 00 1A 00 1E 00 1E 00 1C 00 1C 00 1F 00 20 00 22 00 28 00 2E 00 2F 00 2C 00 2E 00 34 00 33 00 31 00 34 00 39 00 3A 00 37 00 3C 00 42 00 41 00 3D 00 3D 00 41 00 40 00 3C 00 40 00 44 00 47 00 45 00 44 00 45 00 41 00 3A 00 37 00 33 00 30 00 2B 00 29 00 2B 00 28 00 23 00 1F 00 1E 00 1C 00	

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (3B yte)	VALUE (977Byte)	CR C (2B yte)
								17 00 18 00 1A 00 17 00 12 00 10 00 11 00 0D 00 06 00 05 00 07 00 02 00 FB FF FC FF 01 00 FE FF F6 FF F3 FF F3 FF EE FF E5 FF E1 FF E1 FF DD FF D5 FF D1 FF CF FF CA FF C3 FF C2 FF C2 FF BD FF B1 FF AF FF B2 FF AE FF AA FF AA FF B2 FF B1 FF A9 FF A7 FF A9 FF A8 FF A2 FF 9F FF A4 FF A4 FF 9F FF 9F FF A2 FF A2 FF 9F FF A2 FF AA FF B0 FF B1 FF B5 FF BE FF C4 FF C6 FF CA FF CE FF CE FF D0 FF DC FF EB FF F0 FF F4 FF 00 00 0E 00 15 00 14 00 17 00 25 00 2D 00 A8 00 CA FF 95 00 F9 FF 7A 00 24 00 6C 00 57 00 5C 00 76 00 5E 00 90 00 62 00 8F 00 68 00 8B 00 6E 00 88 00	

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (3B yte)	VALUE (977Byte)	CR C (2B yte)
								80 00 89 00 80 00 7D 00 86 00 7B 00 7A 00 70 00 73 00 67 00 62 00 61 00 63 00 58 00 4D 00 4F 00 52 00 4B 00 3F 00 3D 00 3F 00 39 00 2D 00 2A 00 2D 00 2C 00 20 00 1C 00 18 00 0E 00 01 00 F9 FF FB FF F8 FF F1 FF F1 FF F1 FF F0 FF EB FF E8 FF EB FF E0 FF D5 FF D6 FF D7 FF D5 FF D1 FF D7 FF E5 FF DD FF CC FF C2 FF C1 FF BB FF AE FF AB FF B1 FF AF FF A7 FF AB FF B7 FF B6 FF AA FF A6 FF AB FF AC FF A3 FF A3 FF A8 FF 9D FF 94 FF 9B FF A7 FF A8 FF 9E FF 9E FF A3 FF 99 FF 90 FF 91 FF 9C FF A0 FF A0 FF AB FF BA FF BF FF BF FF BE FF C6 FF CA FF C6 FF CE FF DD FF E9 FF	

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (3B yte)	VALUE (977Byte)	CR C (2B yte)
								F2 FF FA FF 0D 00 13 00 0F 00 1C 00 27 00 20 00 17 00 20 00 33 00 2C 00 1A 00 20 00 30 00 35 00 2E 00 3B 00 51 00 4A 00 31 00 2C 00 32 00 29 00 14 00 0A 00 10 00 0D 00 02 00 FF FF 08 00 08 00 01 00 FB FF FE FF 08 00 08 00 12 00 22 00 21 00 23 00 39 00 57 00 66 00 6F 00 95 00 BD 00 C7 00 C4 00 DA 00 FE	

说明

Payload 中的 length 默认为 “02”，由于 “02” 的 bit7 为 0 表示 length 包含子节点，表明后续 2 个字节的长度为 value 的长度。

1.4.5.4 传输通知帧

1.4.5.4.1 传输通知帧字段详解

表1-42 传输通知字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
transmit	0x01	sizeof(tra	struct	• uint32_t	M	1.0/1.0

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
_notify		nsmit_sta te_notify_ pkt_t) + sizeof(dat a)		transmit_id: 单次传输唯一 标识。 - uint32_t state_code: 详见枚举 transmit_stat e_notify_ code_ _t。 - uint32_t len: 数据内容大小 值。单位: Byte。 - uint8_t data[0]: 数据 内容。		

```
typedef enum {  
    TRANSMIT_STATE_NOTIFY_INVALID_ID = 10, /* transmit id无效 */  
    TRANSMIT_STATE_NOTIFY_FINISH,          /* 传输结束 */  
    TRANSMIT_STATE_NOTIFY_FINISH_2,         /* 传输结束, 暂未使用 */  
    TRANSMIT_STATE_NOTIFY_DUPLICATE_ID,     /* transmit id重复, 暂未使用 */  
    TRANSMIT_STATE_NOTIFY_STOP_ID = 16,     /* 传输中止 */  
    TRANSMIT_STATE_NOTIFY_SAVE_FAILED_ID,   /* 传输出错 */  
} transmit_state_notify_code_t;
```

1.4.5.4.2 传输通知帧结构及示例

Model ID 0x06	Command ID 0x05	transmit_notify TLV
------------------	--------------------	------------------------

传输通知帧示例:
AF 03 13 00 42 06 05 81 8C 01 00 00 00 0B 00 00 00 00 00 00 00 A7 EE

表1-43 传输通知帧示例

SOF (1Byte)	CTRL (1Byte)	IENGTH (2Byte)	FID (1Byte)	MODEL_ID (1Byte)	COMMAND_ID (1Byte)	TYPE (1Byte)	LEN (1Byte)	VALUE (12Byte)	CRC (2Byte)
AF	03	1300	42	06	05	81	8C	01 00 00 00 0B 00 00 00 00 00 00 00	A7EE

1.4.5.5 传输停止帧

1.4.5.5.1 传输停止帧字段详解

表1-44 传输停止帧字段描述

字段名称	Type (bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
transmit_stop	0x01	sizeof(transmit_stop_pkt_t)	struct	- uint32_t transmit_id: 单次传输唯一标识。 - uint32_t reason: 预留，默认为0。	O	1.0/1.0

1.4.5.5.2 传输停止帧结构及示例

Model ID 0x06	Command ID 0x06	transmit_stop TLV
------------------	--------------------	----------------------

传输停止帧示例:

AF 03 0F 00 24 06 06 81 88 01 00 00 00 00 00 00 00 E0 88

表1-45 传输停止帧示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE (2Byte)	CR C (2B yte)
AF	03	0F 00	24	06	06	81	88	01000000 00 00 00 00	E0 88

1.4.5.6 传输列举帧

1.4.5.6.1 传输列举帧字段详解

表1-46 传输列举帧字段描述

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类 型	说明	必选/可选 (M/O)	版本
transmit _ls	0x01	sizeof(dia g_ls_cmd _t) + sizeof (path)	struct	- uint32_t dir_len: strlen(path)+1 - uint8_t path[0]: 路径 名称, 要带结 束符, 对应长 度为路径名长 度+1 字节结 束符。	O	1.0/1.0
transmit _ls_ack	0x02	sizeof(dia g_ls_ind_ t + sizeof(file _name)	struct	- uint16_t idx: 文件序号。 - uint8_t path_len: strlen(file name)+1。 - uint8_t	O	1.0/1.0

字段名称	Type(bit0 ~ bit6)	Length Size(Oct)	Value 数据类型	说明	必选/可选 (M/O)	版本
				file_type: 文件类型, 文件为 0, 文件夹为 1。 • uint32_t file_size: 文件大小, 单位: Byte。 • uint8_t file_name[0]: 文件名, 对应长度为路径名长度+1 字节结束符。		

1.4.5.6.2 传输列举帧结构及示例

Model ID 0x06	Command ID 0x07	transmit_Is TLV
------------------	--------------------	--------------------

传输列举帧示例:

AF 03 1D 00 24 06 07 81 96 12 00 00 00 2F 75 73 65 72 2F 6C 6F 67 2F 78 69 61 6F 64 75 2F 00 45 0C

表1-47 传输列举帧示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE (22Byte)	CR C (2B yte)
AF	03	1D 00	24	06	07	81	96	12 00 00 00 2F 75 73 65 72 2F 6C 6F	45 0C

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE (22Byte)	CR C (2B yte)
								67 2F 78 69 61 6F 64 75 2F 0	

1.4.5.6.3 传输列举应答帧示例

传输列举帧示例：

- 列举文件帧

收到第 1 条帧：

AF 03 1B 00 32 06 07 82 94 00 00 0C 01 00 00 00 00 61 75 74 6F 5F 63 72 65
61 74 65 00 7F 53

收到第 2 条帧：

AF 03 29 00 32 06 07 82 A2 01 00 1A 00 C2 45 06 00 32 30 32 33 2D 31 32 2D
31 33 2D 31 37 57
52 54 4D 5F 4C 6F 67 2E 6C 6F 67 00 83 CD

收到第 3 条帧：

AF 03 20 00 32 06 07 82 99 02 00 11 00 08 98 05 00 43 44 49 6D 61 67 65 5F
31 31 32 4B 2E 6D 70 33 00 0B 7E

- 列举结束帧

AF 03 0F 00 32 06 07 82 88 FF FF FF FF FF FF FF 8B 99

表1-48 传输列举应答帧示例

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE	CR C (2B yte)
AF	03	1B 00	32	06	07	82	94	00 00 0C 01 00 00 00 00 61 75 74 6F 5F 63 72 65 61 74 65 00	4D F4

SO F (1B yte)	CT RL (1B yte)	IEN GT H (2B yte)	FID (1B yte)	MODE L_ID (1Byte)	COMMA ND_ID (1Byte)	TYP E (1B yte)	LEN (1B yte)	VALUE	CR C (2B yte)
AF	03	29 00	32	06	07	82	A2	01 00 1A 00 C2 45 06 00 32 30 32 33 2D 31 32 2D 31 33 2D 31 37 57 52 54 4D 5F 4C 6F 67 2E 6C 6F 67 00	83 CD
AF	03	20 00	32	06	07	82	99	02 00 11 00 08 98 05 00 43 44 49 6D 61 67 65 5F 31 31 32 4B 2E 6D 70 33 00	0B 7E
AF	03	0F 00	32	06	07	82	88	(8byte)FF FF FF FF FF FF FF FF	8B 99